МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

**ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ**

МЕТОДИЧНІ ВКАЗІВКИ

**до лабораторних робіт з дисципліни
«ТЕХНОЛОГІЇ ВЕБ-РОЗРОБКИ»**

Електронне видання

Харків 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторних робіт з дисципліни
«ТЕХНОЛОГІЇ ВЕБ-РОЗРОБКИ»

для здобувачів 1-го (бакалаврського) рівня вищої освіти
спеціальності F7 «Комп’ютерна інженерія»
освітньої програми «Комп’ютерна інженерія»

Електронне видання

ЗАТВЕРДЖЕНО
кафедрою АПОТ
Протокол № __
від __.__.2026

Харків 2026

Методичні вказівки до лабораторних робіт з дисципліни «Технології веб-розробки» для для здобувачів 1-го (бакалаврського) рівня вищої освіти спеціальності F7 «Комп’ютерна інженерія» освітньої програми «Комп’ютерна інженерія» [Електронний ресурс] / Упоряд.: В.В. Безродний. − Електронне видання. − Харків: ХНУРЕ, 2026. − 68 с. – pdf

Упорядник В.В. Безродний, асистент каф. АПОТ

Рецензент: О.С. Шкіль, доцент каф. АПОТ, к.т.н., с.н.с.

ЗМІСТ

ВСТУП..... 5
1 ОСНОВИ JAVASCRIPT ТА ДИНАМІЧНА ВЗАЄМОДІЯ З DOM.....6
2 АСИНХРОННА ВЗАЄМОДІЯ ІЗ API.....28
3 СТВОРЕННЯ КОМПОНЕНТІВ НА REACT.....41
4 УПРАВЛІННЯ СТАНОМ ЧЕРЕЗ REDUX.....54

ВСТУП

Веб-технології є однією з ключових складових професійної підготовки фахівців з комп'ютерної інженерії. Сучасний веб-застосунок — це не статичний набір гіпертекстових документів, а повноцінне клієнтське програмне забезпечення, що виконується у браузері, асинхронно взаємодіє з віддаленими сервісами та керує власним станом. Відповідно змінилися й вимоги до розробника: окрім володіння засобами розмітки й стилізації, він має впевнено працювати з мовою JavaScript, компонентною архітектурою інтерфейсу та підходами до централізованого управління станом застосунку.

Методичні вказівки призначені для організації та виконання лабораторних робіт з дисципліни «Технології веб-розробки» здобувачами першого (бакалаврського) рівня вищої освіти спеціальності F7 «Комп'ютерна інженерія» освітньої програми «Комп'ютерна інженерія». Їх мета — сформувати практичні навички проєктування та реалізації клієнтської частини веб-застосунків і забезпечити досягнення компетентностей і програмних результатів навчання, визначених освітньою програмою та робочою програмою дисципліни.

Особливістю лабораторних робіт є її наскрізний характер. Обравши варіант предметної області під час виконання першої лабораторної роботи, здобувач працює над ним і надалі: у третій роботі створений застосунок переробляється засобами React, у четвертій — переводиться на управління станом через Redux. Такий підхід дає змогу простежити еволюцію однієї й тієї самої задачі за різних технологічних рішень і завершити семестр цілісним проєктом. Друга лабораторна робота виконується як окремий навчальний застосунок, присвячений асинхронній взаємодії з відкритими API.

Кожна робота має уніфіковану структуру: мета роботи, методичні вказівки з організації самостійної роботи здобувачів, теоретичні відомості, покроковий порядок виконання, індивідуальні завдання (двадцять основних варіантів і додаткові завдання на підвищену оцінку), вимоги до змісту звіту та контрольні запитання для самоперевірки. Результати виконання й захисту лабораторних робіт ураховуються під час підсумкового контролю відповідно до робочої програми дисципліни.

Для виконання робіт необхідні: сучасний браузер із вбудованими інструментами розробника, редактор коду (наприклад, Visual Studio Code), середовище Node.js із пакетним менеджером npm, а також розширення React Developer Tools і Redux DevTools; роль навчального REST API виконує json-server.

Набуті вміння є підґрунтям для вивчення дисципліни «Технології back-end розробки веб-застосувань», у межах якої навчальний макет API замінюється реальним серверним застосунком.

1 ОСНОВИ JAVASCRIPT ТА ДИНАМІЧНА ВЗАЄМОДІЯ З DOM

1.1 Мета роботи

Метою лабораторної роботи є закріплення практичних навичок **семантичної верстки** веб-сторінок засобами **HTML5**, формування вмінь створення адаптивних макетів інтерфейсу за допомогою технологій **CSS3** (**Flexbox** та **Grid Layout**), отримання базових навичок об'єктно-орієнтованого програмування мовою **JavaScript ES6+** — створення класів, моделювання сутностей предметної області та реалізація логіки управління колекцією об'єктів, а також набуття практичних навичок динамічної модифікації вмісту сторінки засобами **DOM** та обробки **подій** користувача.

Окремим завданням роботи є формування вмінь використання **регулярних виразів (RegExp)** для клієнтської валідації, пошуку та форматування даних, що вводяться користувачем у форму, а також засвоєння підходів до збереження стану застосунку на стороні клієнта за допомогою **localStorage** у форматі **JSON**.

1.2 Методичні вказівки з організації самостійної роботи здобувачів

Самостійна підготовка здобувача до виконання лабораторної роботи передбачає попереднє опрацювання теоретичного матеріалу відповідних лекцій дисципліни («Основи HTML та CSS», «Основи JavaScript», «DOM. Події», «Форми та інтерактивні елементи»), а також практичне ознайомлення із засобами розробки клієнтських веб-застосунків.

Під час самостійної підготовки здобувачу необхідно:

- повторити структуру HTML5-документа, семантичні теги та елементи форм;
- ознайомитися з моделлю блокової верстки (Box Model) та принципами побудови макетів за допомогою Flexbox і Grid Layout;
- повторити основи синтаксису JavaScript ES6+ (оголошення змінних, типи даних, функції, класи);
- опрацювати методи роботи з DOM-деревом (пошук, створення, вставлення та видалення елементів) і модель обробки подій у браузері, зокрема механізм спливання подій;
- опрацювати теоретичний матеріал щодо побудови та застосування регулярних виразів;

- розглянути приклади типових патернів валідації (email, номер телефону, пароль, дата).
- Рекомендовані джерела для самостійного опрацювання:
- **MDN Web Docs** — офіційна документація з HTML, CSS, JavaScript та DOM API (розділи «Guide», «Reference»);
- **uk.javascript.info** — розділи «Основи JavaScript», «Об'єкти: основне», «Класи», «Документ», «Вступ до подій», «Форми та інтерактивні елементи», «Регулярні вирази»;
- **W3Schools** — практичні приклади та онлайн-тренажери з HTML/CSS/JS, роботи з формами та подіями;
- підручник **Eloquent JavaScript** (Marijn Haverbeke) — розділи щодо структур даних та об'єктів, а також глави 9 (регулярні вирази), 14 (DOM) і 18 (форми);
- **regex101.com** — онлайн-середовище для побудови й перевірки регулярних виразів.

Перед виконанням роботи варто переглянути приклади готових валідаторів форм, навчитися писати базові регулярні вирази для перевірки e-mail, номерів телефонів, дат та інших типових шаблонів, а також опанувати прийоми динамічного оновлення сторінки у відповідь на дії користувача. Опрацювання зазначених джерел дозволить не лише виконати лабораторну роботу, а й сформувати базові навички створення динамічного інтерфейсу користувача, що є ключовими для сучасного фронтенд-розробника.

Правила підготовки структури файлів. Перед початком практичної частини здобувач повинен створити окрему проєктну теку з назвою, що відображає обрану предметну область. Усередині теки код обов'язково розподіляється на файли за призначенням: розмітка (index.html), стилі (styles.css) та логіка (app.js, або декілька модульних файлів, наприклад models.js і app.js). Використання зовнішніх бібліотек чи фреймворків на цьому етапі не допускається — усі рішення реалізуються засобами нативного (vanilla) JavaScript.

1.3 Опис лабораторної установки

Лабораторна установка являє собою веб-сторінку з формою введення даних та елементами інтерфейсу, які динамічно оновлюються засобами JavaScript без перезавантаження сторінки. Для виконання роботи використовується стандартне

програмно-апаратне середовище для клієнтської веб-розробки. Здобувачу необхідні:

- персональний комп'ютер з установленим браузером **Google Chrome** (або іншим сучасним браузером на основі Chromium) з підтримкою вбудованих інструментів розробника **Chrome DevTools** для інспекції DOM-дерева, перевірки стилів, консолі помилок, відлагодження скриптів та контролю обробки подій;
- редактор коду **Visual Studio Code** з установленим розширенням **Live Server** для автоматичного оновлення сторінки браузера під час редагування файлів;
- за потреби — онлайн-сервіс для тестування регулярних виразів (наприклад, regex101.com), а також розширення VS Code для перевірки синтаксису (лінтери), автодоповнення HTML/CSS/JS та форматування коду.

Додаткового серверного середовища чи мережевого з'єднання для виконання цієї роботи не потрібно: застосунок є повністю клієнтським та запускається локально через Live Server.

1.4 Порядок виконання роботи і методичні вказівки з її виконання

1.4.1 Теоретична частина

Семантичні теги HTML5

Семантичні теги — це елементи HTML5, назва яких відображає смислове призначення вмісту, що робить розмітку зрозумілішою як для розробників, так і для браузерів і пошукових систем. До основних семантичних тегів належать:

- <header> — верхній колонтитул сторінки або розділу (логотип, заголовок, навігація);
- <nav> — блок навігаційних посилань;
- <main> — основний унікальний вміст сторінки (використовується один раз);
- <section> — тематичний розділ документа;
- <article> — самостійний, незалежний блок контенту;
- <aside> — бічна панель або додаткова інформація;
- <footer> — нижній колонтитул сторінки або розділу.

Приклад базового семантичного каркасу сторінки:

```
<body>
  <header>
    <h1>Назва застосунку</h1>
```

```
    <nav><!-- навігаційні посилання --></nav>
  </header>
  <main>
    <section class="add-form-section">
      <!-- форма додавання сутності -->
    </section>
    <section class="entity-list-section">
      <!-- контейнер списку, що наповнюється
           динамічно засобами DOM -->
      <ul id="entity-list"></ul>
    </section>
  </main>
  <footer><p>&copy; 2026</p></footer>
</body>
```

Елементи форм та атрибути валідації

Форми будуються за допомогою тега <form>, що об'єднує елементи введення: <input>, <select>, <textarea>, <button>. Кожне поле <input> має атрибут type, який визначає тип введених даних (text, email, tel, number, password тощо).

HTML5 надає вбудовані **атрибути валідації**, які браузер перевіряє автоматично ще до виконання JavaScript-коду:

- required - поле обов'язкове для заповнення;
- pattern - значення повинно відповідати заданому регулярному виразу;
- minlength / maxlength - мінімальна / максимальна довжина рядка;
- min / max - межі числового значення.

Приклад поля з базовою HTML5-валідацією:

```
<label for="email">E-mail:</label>
<input type="email" id="email" name="email" required
       pattern="^[^\s@]+@[^\s@]+\.[^\s@]{2,}$">
```

Важливо розуміти, що вбудована HTML5-валідація є лише першим рубежем перевірки: у практичній частині роботи додатково реалізується **JavaScript-валідація** за допомогою регулярних виразів, яка дає змогу гнучкіше керувати повідомленнями про помилки та стилізацією полів.

Основи синтаксису JavaScript ES6+

Змінні оголошуються ключовими словами let (значення, що може змінюватися) та const (константа). Використання застарілого var не рекомендується.

Примітивні типи даних у JavaScript: String, Number, Boolean, undefined, null, Symbol, BigInt.

Стрілкові функції (arrow functions) — компактний синтаксис оголошення функцій:

```
const isValidLength = (value, min) =>
  value.trim().length >= min;
```

Класи (class) — синтаксичний засіб ES6+ для реалізації об'єктно-орієнтованого підходу. Клас містить конструктор (constructor) для ініціалізації властивостей об'єкта та методи для опису його поведінки:

```
class Item {
  constructor(title, category) {
    this.id = crypto.randomUUID();
    this.title = title;
    this.category = category;
    this.createdAt = new Date().toISOString();
  }
}

class ItemManager {
  constructor() {
    this.items = [];
  }
  add(item) {
    this.items.push(item);
  }
  remove(id) {
    this.items = this.items.filter(i => i.id !== id);
  }
}
```

Клас-**менеджер** (наприклад, ItemManager) інкапсулює масив об'єктів колекції та методи роботи з ним, тоді як клас-**сутність** (наприклад, Item) описує окремий об'єкт предметної області.

Регулярні вирази (RegExp)

Регулярний вираз — це шаблон, який описує множину рядків із певною структурою. В JavaScript регулярний вираз створюється або літералом (/шаблон/прапорці), або через конструктор new RegExp().

Основні елементи шаблонів:

- **символьні класи** — \d (цифра), \w (символ слова), \s (пробільний символ), а також власні набори [a-zA-Z0-9_];
- **квантифікатори** — + (один або більше), * (нуль або більше), ? (нуль або один), {n,m} (від n до m повторень);

- **якорі** — ^ (початок рядка), $ (кінець рядка);
- **альтернація** — | (логічне «АБО»).
- Основні методи роботи з регулярними виразами:
- regex.test(text) — перевіряє, чи відповідає рядок заданому шаблону (повертає true або false);
- text.match(/regex/g) — повертає знайдені у тексті збіги із шаблоном;
- text.replace(/regex/g, 'заміна') — замінює частину тексту, що відповідає шаблону (застосовується, зокрема, для підсвічування результатів пошуку та автоматичного форматування введених даних).

Приклади типових шаблонів валідації:

```
const emailPattern =
  /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i;
const phonePattern =
  /^\+?\d{1,3}[\s-]?\(?\d{2,3}\)?[\s-]?\d{3}[\s-]?\d{2}[\s-]?\d{2}$/;
const passwordPattern =
  /^(?=.*[A-Za-z])(?=.*\d).{8,}$/;
console.log(emailPattern.test("student@nure.ua")); // true
```

Робота з DOM (Document Object Model)

DOM — це програмний інтерфейс, що представляє HTML-документ у вигляді дерева об'єктів, доступних для читання та зміни засобами JavaScript. Основні методи й властивості роботи з DOM:

- document.querySelector(selector) / document.querySelectorAll(selector) — пошук елемента (або колекції елементів) за CSS-селектором;
- document.createElement(tagName) — створення нового елемента;
- parent.append(child) / parent.appendChild(child) — вставлення дочірнього елемента;
- element.remove() / parent.removeChild(child) — видалення елемента з дерева;
- element.textContent та element.innerHTML — читання і зміна текстового або HTML-вмісту елемента;

- element.setAttribute(name, value) та element.dataset — робота з атрибутами, зокрема з користувацькими атрибутами data-* (наприклад, data-id для зв'язку елемента розмітки з об'єктом колекції);
- element.classList.add() / remove() / toggle() — керування класами елемента.

Динамічне формування інтерфейсу означає, що елементи колекції не описуються заздалегідь у файлі розмітки, а створюються засобами JavaScript на основі масиву об'єктів. Типовим рішенням є **функція рендерингу**, яка очищує контейнер і повторно формує його вміст після кожної зміни даних — саме такий підхід використовується у практичній частині цієї роботи.

Обробка подій

Подія — це сигнал браузера про те, що на сторінці відбулася певна дія: click, input, change, submit, keydown, focus, blur тощо. Обробник приєднується методом element.addEventListener('подія', функція). Об'єкт події (event), який передається обробнику, містить такі важливі складові:

- event.target — елемент, на якому подія виникла;
- event.currentTarget — елемент, до якого приєднано обробник;
- event.preventDefault() — скасування стандартної дії браузера (наприклад, перезавантаження сторінки під час надсилання форми).

Оскільки події **спливають** (bubbling) від вкладеного елемента до батьківських, для динамічно створених елементів застосовують **делегування подій**: один обробник приєднується до спільного контейнера, а конкретний елемент, що спричинив подію, визначається через event.target.closest(selector). Такий підхід не потребує повторного приєднання обробників після кожного перерендерингу списку і є обов'язковим для виконання цієї лабораторної роботи.

Робота з формами

Елемент <form> є основним засобом отримання введених користувачем даних. Ключові властивості та методи:

- input.value — поточне значення текстового поля;
- checkbox.checked — стан прапорця (обраний / не обраний);
- form.elements — колекція всіх полів форми;
- об'єкт FormData — засіб отримання значень усіх полів форми одночасно;

- подія submit, що виникає під час надсилання форми, — у поєднанні з event.preventDefault() дозволяє реалізувати власну логіку обробки даних без перезавантаження сторінки.

1.4.2 Практична частина

Крок 1. Створення структури проєкту. Створити проєктну теку та в ній три базові файли: index.html, styles.css, app.js. За потреби (наприклад, для розділення класів моделей і логіки застосунку) допускається модульна структура — окремі файли на кшталт models.js.

Крок 2. Проєктування семантичної розмітки сторінки. Відповідно до обраного варіанта (розділ 1.5.2) розробити семантичну структуру index.html: шапку сторінки (<header>) з назвою застосунку, за потреби — навігацію (<nav>), форму додавання сутності (<form>) з усіма полями, необхідними для обраної предметної області, панель пошуку та фільтрації, а також порожній семантичний контейнер (<section>, <ul> або <table>) під список записів. Важливо: елементи колекції у розмітці не описуються — вони формуються динамічно засобами DOM.

Крок 3. Стилізація інтерфейсу за допомогою CSS. Розробити файл styles.css, застосовуючи **Flexbox** або **Grid Layout** для побудови адаптивного макета сторінки (розташування шапки, форми, панелі фільтрів та контейнера списку). Стилізувати елементи форми та картки/рядки списку. Передбачити окремі класи для станів інтерфейсу: індикації помилки валідації (наприклад, .error або .invalid — червона рамка, підказка з текстом помилки) та особливих станів записів (наприклад, .done, .expired, .selected).

Крок 4. Створення базових JavaScript-класів предметної області. У файлі логіки реалізувати щонайменше два класи ES6+: клас **сутності** обраного варіанта (властивості з конструктора, унікальний ідентифікатор) та клас-**менеджер**, що інкапсулює масив об'єктів і методи роботи з колекцією — додавання, редагування, видалення за ідентифікатором, пошук і фільтрацію.

Крок 5. Додавання перевірки форми. Додати обробник події submit форми, обов'язково викликаючи event.preventDefault(), щоб запобігти стандартній відправці та перезавантаженню сторінки. Отримати значення полів (через властивість .value або об'єкт FormData) і перевірити кожне поле відповідним регулярним виразом. У разі невідповідності — додати клас помилки до поля та вивести текст підказки; у разі успіху — прибрати індикацію помилки.

Крок 6. Збереження створеного об'єкта в масив та localStorage. Після успішної валідації створити новий екземпляр класу сутності, додати його до масиву класу-менеджера, після чого серіалізувати весь масив у рядок за допомогою JSON.stringify() та зберегти в localStorage. При завантаженні сторінки виконати зворотну дію: зчитати збережений рядок методом localStorage.getItem(),

перетворити його назад у масив об'єктів методом JSON.parse() та ініціалізувати ним масив менеджера — це забезпечить збереження даних застосунку між перезавантаженнями сторінки.

```
// збереження
localStorage.setItem("items",
  JSON.stringify(manager.items));

// відновлення при завантаженні сторінки
const saved = localStorage.getItem("items");
if (saved) {
  manager.items = JSON.parse(saved);
}
```

Крок 7. Динамічне відображення колекції засобами DOM. Реалізувати функцію рендерингу, яка отримує масив об'єктів, очищує контейнер списку та для кожного об'єкта створює елемент розмітки методом document.createElement(). Ідентифікатор об'єкта зберігати в атрибуті data-id, щоб пов'язати елемент інтерфейсу з відповідним записом колекції. Функцію рендерингу викликати після завантаження сторінки та після кожної зміни даних.

```
const list = document.querySelector("#entity-list");

function render(items) {
  list.innerHTML = "";
  items.forEach((item) => {
    const li = document.createElement("li");
    li.className = "entity-item";
    li.dataset.id = item.id;
    li.textContent = item.title;

    const delBtn = document.createElement("button");
    delBtn.className = "delete-btn";
    delBtn.textContent = "Видалити";

    li.append(delBtn);
    list.append(li);
  });
}
```

Крок 8. Обробка дій користувача над записами. Реалізувати редагування та видалення записів. Оскільки елементи списку створюються динамічно, обробник події приєднується не до кожної кнопки, а до контейнера списку — за принципом делегування подій. Потрібний елемент визначається методом

event.target.closest(), після чого виконується відповідна операція над колекцією, оновлюється localStorage та викликається повторний рендеринг.

```
list.addEventListener("click", (event) => {
  const btn = event.target.closest(".delete-btn");
  if (!btn) return;

  const id = btn.closest(".entity-item").dataset.id;
  manager.remove(id);
  saveToStorage();
  render(manager.items);
});
```

Крок 9. Пошук, фільтрація та сортування у реальному часі. Додати обробники подій input (поле пошуку) та change (списки фільтрів і сортування). Результат обчислюється як похідний масив методами filter() і sort() від основної колекції — вихідні дані при цьому не змінюються, — після чого відфільтрований масив передається у функцію рендерингу. Для пошуку за фрагментом тексту доцільно застосувати регулярний вираз, побудований конструктором new RegExp().

```
searchInput.addEventListener("input", (event) => {
  const query = event.target.value.trim();
  const pattern = new RegExp(query, "i");
  const found = manager.items
    .filter((item) => pattern.test(item.title));
  render(found);
});
```

Крок 10. Тестування та налагодження застосунку. Перевірити коректність роботи всіх компонентів сторінки за допомогою Chrome DevTools: відповідність введених даних очікуваному формату, коректність оновлення DOM після кожної операції, відсутність помилок у консолі, збереження стану після перезавантаження сторінки. Окремо перевірити граничні випадки: порожня колекція, порожні або некоректні значення полів, дублювання записів, пошук без результатів. За результатами тестування — оптимізувати код і покращити логіку реалізації.

1.5 Завдання

Загальні вимоги до всіх варіантів:

- розподіл коду на окремі файли: HTML — для структури, CSS — для стилізації, JavaScript — для логіки;

- використання виключно нативного (vanilla) JavaScript, без сторонніх бібліотек і фреймворків (React, jQuery тощо);
- семантична верстка сторінки з використанням щонайменше 4—5 семантичних тегів HTML5;
- адаптивна стилізація інтерфейсу засобами Flexbox або Grid Layout;
- реалізація щонайменше двох класів ES6+ — класу сутності та класу-менеджера колекції;
- валідація різних типів полів форми за допомогою регулярних виразів (наприклад, email, телефон, пароль, дата, числове значення) з візуальною індикацією помилки;
- коректна обробка події submit із застосуванням event.preventDefault();
- динамічне формування списку записів (карток, рядків таблиці, елементів сітки) засобами DOM — елементи колекції не описуються статично у файлі розмітки;
- реалізація додавання, редагування та видалення записів із негайним оновленням інтерфейсу без перезавантаження сторінки;
- обробка щонайменше чотирьох різних типів подій (наприклад, submit, click, input, change, keydown) за допомогою addEventListener() із застосуванням делегування подій для елементів динамічного списку;
- реалізація пошуку, фільтрації або сортування колекції з оновленням результату в реальному часі;
- збереження масиву створених об'єктів у localStorage у форматі JSON та його відновлення під час завантаження сторінки.

Обсяг конкретної реалізації визначається обраним варіантом: додаткові механізми взаємодії (модальні вікна, перетягування елементів, таймери, вкладки, підсвічування знайдених фрагментів тощо) зазначені безпосередньо в описі варіанта та є обов'язковими для нього.

Кожна бригада (до двох здобувачів) або здобувач індивідуально обирає **один** з наведених варіантів за списком у журналі академічної групи. За наявності бригади з двох осіб завдання передбачає розподіл обов'язків між учасниками та подальшу інтеграцію їхньої роботи в єдиний застосунок.

Обраний номер варіанта є наскрізним: та сама предметна область використовується також у лабораторних роботах № 3 і № 4, де розроблений застосунок послідовно переробляється засобами React (ЛР № 3) та переводиться на централізоване управління станом через Redux (ЛР № 4).

Таблиця 1.1 — Варіанти завдань до лабораторної роботи № 1

Варіант та назва	Завдання
Варіант 1. Система управління бібліотекою	**Завдання:** розробити веб-додаток для управління бібліотекою: додавання книги через форму з валідацією полів (назва, автор, рік видання, ISBN), динамічне формування переліку книг у вигляді карток або рядків таблиці засобами DOM, редагування та видалення записів через делегування подій. Реалізувати пошук за назвою чи автором у реальному часі (подія input), фільтрацію за жанром і роком видання (подія change) та сортування списку книг. Стан колекції зберігати в localStorage. **Приклад можливих моделей (класів):** • Book — назва, автор, рік видання, жанр, ISBN, рейтинг тощо; • Library — методи додавання, редагування, видалення, пошуку, фільтрації та сортування книг.
Варіант 2. Менеджер завдань (To-Do List)	**Завдання:** розробити веб-додаток для управління списком завдань: додавання завдання через форму (опис, дедлайн, пріоритет) з валідацією дати регулярним виразом, динамічне відображення списку засобами DOM, позначення виконання кліком (перекреслення тексту через клас done), редагування та видалення записів. Порядок завдань користувач змінює перетягуванням за допомогою Drag and Drop API (dragstart, dragover, drop). Передбачити лічильник активних і виконаних завдань, що оновлюється динамічно. **Приклад можливих моделей (класів):** • Task — опис, статус виконання, дедлайн, пріоритет, порядковий номер тощо;

Варіант та назва	Завдання
	• TaskManager — методи додавання, редагування, видалення, зміни статусу, зміни порядку та фільтрації завдань.
Варіант 3. Система обліку студентів	**Завдання:** розробити веб-додаток для ведення обліку студентів та їхніх оцінок у вигляді динамічної HTML-таблиці, рядки якої створюються й видаляються засобами DOM. Реалізувати додавання студента через форму з валідацією (ПІБ, номер групи, оцінки), автоматичний розрахунок і відображення середнього балу, сортування таблиці за кліком на заголовок колонки, пошук студента за прізвищем у реальному часі та формування списку відмінників за пороговим балом із візуальним підсвічуванням відповідних рядків. **Приклад можливих моделей (класів):** • Student — ПІБ, група, оцінки (масив чисел), середній бал тощо; • StudentManager — методи додавання, редагування, видалення, розрахунку середнього балу, сортування та фільтрації.
Варіант 4. Інтерактивний словник	**Завдання:** створити застосунок для ведення словника: додавання нових слів та їхніх визначень через форму з валідацією, динамічне формування статей словника засобами DOM, редагування та видалення записів. Реалізувати пошук за словом у реальному часі з підсвічуванням знайденого фрагмента в тексті за допомогою регулярних виразів (метод replace) та сортування статей за алфавітом. **Приклад можливих моделей (класів):** • Word — слово, переклад або визначення, приклад використання, категорія тощо; • Dictionary — методи додавання, редагування, видалення, пошуку, підсвічування збігів і сортування статей.
Варіант 5. Калькулятор витрат	**Завдання:** створити веб-додаток для обліку фінансових витрат за категоріями: додавання витрати через форму (сума, категорія, дата) з валідацією суми й

Варіант та назва	Завдання
	дати регулярними виразами, динамічне формування списку операцій та блоку підсумкової статистики засобами DOM. Реалізувати фільтрацію за категорією й періодом, видалення операції з автоматичним перерахунком підсумку, а також індикатор виконання місячного бюджету — смугу прогресу, ширина та колір якої змінюються залежно від суми витрат. **Приклад можливих моделей (класів):** • Expense — сума, категорія, дата, коментар тощо; • ExpenseManager — методи додавання, редагування, видалення, фільтрації за категорією та періодом, розрахунку підсумків.
Варіант 6. Аналізатор тексту	**Завдання:** розробити веб-додаток для аналізу тексту, що підраховує кількість символів, слів і речень у реальному часі під час введення (подія input). За допомогою регулярних виразів побудувати частотний словник слів і вивести його динамічно сформованою таблицею, а також реалізувати пошук фрагмента з підсвічуванням усіх збігів у тексті. Зберігати історію проаналізованих текстів у localStorage з можливістю пошуку в ній, повторного завантаження та видалення записів історії. **Приклад можливих моделей (класів):** • TextAnalyzer — методи підрахунку символів, слів і речень, побудови частотного словника, підсвічування збігів; • HistoryManager — методи збереження, пошуку, відновлення та видалення записів історії.
Варіант 7. Галерея зображень	**Завдання:** розробити застосунок для управління галереєю зображень: додавання зображення за URL через форму з валідацією адреси регулярним виразом, динамічне формування сітки мініатюр засобами DOM (CSS Grid), видалення зображень, пошук за назвою або описом і сортування за датою додавання. Клік по мініатюрі відкриває повнорозмірне зображення в модальному вікні (лайтбокс) із перемиканням

Варіант та назва	Завдання
	клавішами ArrowLeft/ArrowRight та закриттям клавішею Escape або кліком поза зображенням. **Приклад можливих моделей (класів):** • ImageItem — назва, опис, URL зображення, дата додавання, теги тощо; • GalleryManager — методи додавання, видалення, пошуку, сортування зображень та навігації лайтбоксом.
Варіант 8. Калькулятор площі фігур	**Завдання:** розробити веб-додаток для обчислення площі та периметра геометричних фігур. Реалізувати вибір фігури зі списку, після чого набір полів для введення параметрів формується динамічно засобами DOM, а значення перевіряються регулярними виразами (додатні цілі та десяткові числа). Обробку натискань кнопок побудувати через делегування подій на батьківському контейнері з підтримкою введення з клавіатури (keydown). Передбачити динамічний вивід історії обчислень зі збереженням у localStorage. **Приклад можливих моделей (класів):** • Shape — базовий клас із методами обчислення площі та периметра; • Rectangle, Circle, Triangle — класи конкретних фігур (успадковують Shape); • CalculationHistory — методи збереження, відображення та очищення історії обчислень.
Варіант 9. Система тестування	**Завдання:** розробити веб-додаток для створення та проходження тесту. У режимі редагування передбачити динамічне додавання й видалення питань і варіантів відповідей засобами DOM (кнопка «Додати варіант» створює нове поле введення). У режимі проходження питання та варіанти генеруються з масиву даних, обраний варіант підсвічується як правильний або неправильний, рахунок оновлюється динамічно. Реалізувати таймер зворотного відліку (setInterval) з автоматичним переходом до наступного питання та блокуванням зміни відповіді після завершення часу. **Приклад можливих моделей (класів):**

Варіант та назва	Завдання
	• Question — текст питання, варіанти відповідей, правильна відповідь тощо; • Test — список питань, таймер, перевірка відповідей, підрахунок результату; • TestManager — методи створення, збереження, запуску та перевірки тестів.
Варіант 10. Система управління контактами	**Завдання:** розробити веб-додаток для збереження та керування контактами: додавання контакту через форму з валідацією імені, номера телефону та email регулярними виразами, динамічне формування списку контактів засобами DOM із групуванням за категорією (родина, робота, друзі). Реалізувати пошук за ім'ям або номером у реальному часі (подія input), редагування контакту в модальному вікні та видалення записів через делегування подій із підтвердженням дії. **Приклад можливих моделей (класів):** • Contact — ім'я, номер телефону, email, група, нотатки тощо; • ContactManager — методи додавання, редагування, видалення, пошуку та групування контактів.
Варіант 11. Каталог фільмів та серіалів	**Завдання:** розробити веб-додаток для ведення каталогу переглянутих і запланованих до перегляду фільмів та серіалів: додавання запису через форму (назва, жанр, рік випуску, режисер, статус перегляду) з валідацією, динамічне формування карток каталогу засобами DOM, зміна статусу перегляду та видалення записів. Особистий рейтинг реалізувати віджетом із п'яти зірок: наведення курсора (mouseover) підсвічує зірки до поточної позиції, клік фіксує оцінку, знесення курсора (mouseout) повертає збережене значення. Додати фільтрацію за жанром і сортування за рейтингом. **Приклад можливих моделей (класів):** • Movie — назва, жанр, рік випуску, режисер, рейтинг, статус перегляду тощо; • MovieCatalog — методи додавання, редагування, видалення, зміни рейтингу, фільтрації та сортування.

Варіант та назва	Завдання
Варіант 12. Трекер щоденних звичок	**Завдання:** створити застосунок для відстеження щоденних звичок користувача (читання, спорт, вода тощо): додавання звички через форму з валідацією, динамічне формування сітки днів (тиждень або місяць) засобами DOM, де клік по клітинці перемикає позначку виконання, а обробку кліків побудовано на делегуванні подій. Передбачити автоматичний підрахунок поточної серії виконання (streak) та відсотка виконання за період з оновленням статистики в реальному часі, а також підказку з датою при наведенні курсора на клітинку. **Приклад можливих моделей (класів):** • Habit — назва звички, опис, множина дат виконання, поточна серія тощо; • HabitTracker — методи додавання, редагування, видалення, позначення виконання та розрахунку статистики.
Варіант 13. Система бронювання столиків у ресторані	**Завдання:** розробити веб-додаток для бронювання столиків у ресторані. Форма бронювання має містити валідацію імені, телефону, дати й часу регулярними виразами та крос-перевірку коректності (час бронювання не в минулому). Список бронювань формується динамічно засобами DOM і розподіляється за вкладками (наприклад, за датами), перемикання яких відбувається без перезавантаження сторінки. Передбачити перевірку зайнятості столика на обраний час із виведенням повідомлення про конфлікт, редагування та скасування бронювання. **Приклад можливих моделей (класів):** • Reservation — ім'я клієнта, телефон, дата, час, кількість осіб, номер столика тощо; • Table — номер столика, місткість, статус; • BookingManager — методи додавання, редагування, скасування бронювань і перевірки доступності.
Варіант 14. Міні-платформа	**Завдання:** розробити веб-додаток для ведення особистого блогу: додавання статті через форму (заголовок, текст, теги) з валідацією, динамічне

Варіант та назва	Завдання
для публікації статей (блог)	формування списку публікацій засобами DOM, редагування та видалення статей. Для тексту статті реалізувати підтримку спрощеного Markdown-синтаксису (заголовки #, ##, жирний і курсивний текст, посилання у форматі [текст](URL)) з перетворенням у HTML за допомогою регулярних виразів та живим попереднім переглядом під час набору. Додати пошук за заголовком і фільтрацію за тегом. **Приклад можливих моделей (класів):** • Article — заголовок, текст, дата публікації, теги тощо; • BlogManager — методи додавання, редагування, видалення, пошуку, фільтрації за тегом і форматування тексту.
Варіант 15. Щоденник фітнес-тренувань	**Завдання:** створити застосунок для ведення щоденника тренувань. У формі додавання тренування передбачити динамічне створення й видалення блоків вправ (кнопка «Додати вправу» формує нову групу полів засобами DOM) із валідацією числових значень (підходи, повторення, вага, тривалість). Реалізувати динамічний список історії тренувань, фільтрацію за датою та видом вправи, підрахунок загальної статистики за період з оновленням у реальному часі та індикатор виконання тижневої цілі. **Приклад можливих моделей (класів):** • WorkoutEntry — дата, перелік вправ (підходи, повторення, вага), тривалість тощо; • WorkoutDiary — методи додавання, редагування, видалення, фільтрації та розрахунку статистики.
Варіант 16. Каталог рецептів	**Завдання:** розробити веб-додаток для ведення каталогу кулінарних рецептів. У формі додавання рецепта реалізувати динамічний список інгредієнтів (додавання та видалення полів засобами DOM) із валідацією кількості. Рецепти виводити у вигляді акордеона: клік по заголовку розгортає повний опис (classList.toggle), одночасно розгорнутим може бути лише один рецепт. Додати фільтрацію за категорією

Варіант та назва	Завдання
	(сніданок, обід, десерт тощо), сортування за часом приготування та пошук за назвою або інгредієнтом у реальному часі. **Приклад можливих моделей (класів):** • Recipe — назва, список інгредієнтів, категорія, час приготування, складність, покроковий опис тощо; • RecipeCatalog — методи додавання, редагування, видалення, пошуку, фільтрації та сортування рецептів.
Варіант 17. Планувальник подій	**Завдання:** створити веб-додаток для планування особистих або робочих подій: додавання події через форму (назва, дата, час, місце проведення, тип) з валідацією дати й часу регулярними виразами, динамічне формування списку подій засобами DOM із сортуванням за датою та фільтрацією за типом. Реалізувати систему спливаючих сповіщень (toast): при додаванні, редагуванні чи видаленні події динамічно створюється повідомлення відповідного типу (успіх, помилка, інформація), яке автоматично зникає через заданий час (setTimeout) або за кліком на кнопку закриття. **Приклад можливих моделей (класів):** • Event — назва, дата, час, місце проведення, опис, тип тощо; • EventPlanner — методи додавання, редагування, видалення, сортування та фільтрації подій; • NotificationManager — методи створення та приховування сповіщень.
Варіант 18. Трекер вакансій (пошук роботи)	**Завдання:** розробити веб-додаток для відстеження процесу пошуку роботи у вигляді спрощеної Kanban-дошки з колонками «Подано», «Співбесіда», «Оффер / Відмова». Картки заявок створюються через форму з валідацією (компанія, посада, дата подання, посилання на вакансію) і переміщуються між колонками за допомогою Drag and Drop API (dragstart, dragover, drop) зі зміною статусу заявки. Передбачити видалення картки, пошук за назвою компанії та лічильники заявок у кожній колонці.

Варіант та назва	Завдання
	Приклад можливих моделей (класів): • JobApplication — назва компанії, посада, дата подання, статус, посилання, нотатки тощо; • JobTracker — методи додавання, редагування, зміни статусу, видалення та пошуку заявок.
Варіант 19. Облік клієнтів та тварин ветеринарної клініки	**Завдання:** розробити веб-додаток для ведення обліку тварин-пацієнтів ветеринарної клініки: додавання картки тварини через форму (вид, порода, кличка, ім'я та телефон власника, дати останньої й наступної вакцинації) з валідацією телефону та дат регулярними виразами, динамічне формування списку карток засобами DOM. Реалізувати пошук за кличкою тварини або власником у реальному часі, фільтрацію за видом тварини, сортування за датою наступної вакцинації та автоматичне візуальне виділення карток із простроченою вакцинацією. **Приклад можливих моделей (класів):** • Pet — кличка, вид, порода, ім'я та телефон власника, дати останньої й наступної вакцинації тощо; • ClinicRegistry — методи додавання, редагування, видалення, пошуку, фільтрації та перевірки термінів вакцинації.
Варіант 20. Персональний нотатник (щоденник)	**Завдання:** створити веб-додаток для ведення особистих нотаток або щоденникових записів: додавання запису із заголовком, текстом, датою та тегами через форму з валідацією, динамічне формування списку нотаток засобами DOM, редагування запису безпосередньо в інтерфейсі та видалення з підтвердженням. Реалізувати пошук за заголовком або текстом із підсвічуванням знайдених фрагментів за допомогою регулярних виразів, фільтрацію за тегом (кнопки-теги формуються динамічно) та лічильник символів для поля тексту зі зміною кольору при наближенні до ліміту. **Приклад можливих моделей (класів):** • Note — заголовок, текст, дата створення, теги тощо;

Варіант та назва	Завдання
	• NotebookManager — методи додавання, редагування, видалення, пошуку, підсвічування збігів і фільтрації за тегом.

Здобувач також може за погодженням з викладачем запропонувати власну предметну область, якщо вона передбачає роботу з колекцією однотипних об'єктів та відповідає загальним вимогам розділу 1.5.1. У такому разі погоджена предметна область закріплюється за здобувачем і для лабораторних робіт № 3 та № 4.

Додаткове завдання виконується здобувачем за бажанням, понад основний обсяг роботи, і оцінюється додатковими балами. Обирається один із наведених варіантів; додаткове завдання реалізується в межах того самого застосунку, що й основне завдання, і має відповідати загальним вимогам розділу 1.5.1 (нативний JavaScript, без сторонніх бібліотек).

Таблиця 1.2 — Додаткові завдання на підвищену оцінку

Варіант та назва	Завдання
Варіант 1. Імпорт та експорт даних (File API)	Реалізувати експорт поточного стану колекції у файл формату JSON (створення об'єкта Blob та посилання для завантаження) і зворотний імпорт раніше збереженого файлу назад у застосунок із перевіркою структури отриманих даних та об'єднанням їх із наявною колекцією.
Варіант 2. Теми оформлення (Light / Dark mode)	Реалізувати перемикання між світлою та темною темами оформлення інтерфейсу за допомогою CSS-змінних (custom properties) і класу на елементі <body>. Обрана користувачем тема зберігається в localStorage та застосовується автоматично при наступному відкритті сторінки.
Варіант 3. Модальні вікна	Реалізувати модальне вікно для підтвердження видалення запису або перегляду детальної інформації про об'єкт: напівпрозора фон-підложка, закриття клавішею Escape та кліком поза межами вікна, повернення фокуса на елемент, з якого вікно було відкрито.
Варіант 4. Спливаючі	Реалізувати динамічне створення повідомлень про результат операції (успіх, помилка, інформація) з

Варіант та назва	Завдання
сповіщення (Toast notifications)	відповідним кольором. Повідомлення має автоматично зникати через заданий час (setTimeout) або за кліком на кнопку закриття; передбачити можливість одночасного показу кількох повідомлень.
Варіант 5. Зміна порядку записів (Drag and Drop)	Реалізувати зміну порядку елементів колекції перетягуванням за допомогою подій dragstart, dragover та drop із візуальним виділенням позиції вставлення та збереженням нового порядку записів у localStorage.
Варіант 6. Валідація в реальному часі та надійність пароля	Реалізувати перевірку коректності полів регулярними виразами безпосередньо під час набору тексту (подія input) з миттєвою індикацією стану поля, а також візуальний індикатор надійності пароля («слабкий» / «середній» / «надійний») на основі довжини, наявності цифр, великих літер і спеціальних символів.
Варіант 7. Автоматичне форматування введення	Реалізувати автоматичне форматування значення поля під час набору (подія input) без використання сторонніх бібліотек масок: додавання дужок і дефісів у номері телефону, розділювачів розрядів у сумі коштів або роздільників у даті.
Варіант 8. Допоміжні елементи полів форми	Реалізувати спливаючі текстові підказки біля полів форми, що з'являються при отриманні фокуса (подія focus) і зникають при його втраті (подія blur) з динамічним позиціонуванням відносно поля, а також лічильник символів для textarea з обмеженням довжини та зміною кольору при наближенні до ліміту.
Варіант 9. Копіювання даних (Clipboard API)	Реалізувати кнопку копіювання даних запису (наприклад, контакту, посилання або підсумкового результату) у буфер обміну за допомогою Clipboard API (navigator.clipboard.writeText) із тимчасовим візуальним підтвердженням успішного копіювання.
Варіант 10. Крос-валідація пов'язаних полів форми	Реалізувати перевірку взаємної відповідності двох пов'язаних полів — збіг пароля та підтвердження пароля, коректність діапазону дат (дата початку не пізніше дати завершення) або відповідність суми залишку — з виведенням повідомлення про помилку та блокуванням надсилання форми.

1.6 Зміст звіту

Звіт з лабораторної роботи оформлюється у вигляді окремого документа та повинен містити такі обов'язкові розділи:

1. **Титульний аркуш** — повна назва навчального закладу, назва дисципліни, номер і назва лабораторної роботи, прізвище та ім'я здобувача (або членів бригади), номер академічної групи.
2. **Мета роботи** — стисле формулювання мети та завдань, поставлених перед виконанням роботи (відповідно до розділу 1.1 цих методичних вказівок).
3. **Хід роботи**, у якому обов'язково зазначається:

- номер обраного варіанта завдання та, за наявності, номер виконаного додаткового завдання; для бригади — опис розподілу обов'язків між учасниками;
- лістинги ключових фрагментів коду (HTML-розмітки, CSS-стилів, класів ES6+, функції валідації, функції динамічного формування списку та обробників подій) з поясненнями, що описують логіку реалізації;
- скріншоти інтерфейсу користувача: загальний вигляд сторінки, сформований засобами DOM список записів, стан форми під час коректного заповнення та у стані помилки валідації (з підсвіченими червоним полями), а також результат роботи пошуку або фільтрації.

4. **Висновки** — узагальнення отриманих результатів, самооцінка виконаної роботи, труднощі, з якими зіткнувся здобувач під час виконання, та можливі шляхи вдосконалення реалізованого функціоналу.

1.7 Контрольні запитання і завдання

1. Які семантичні теги HTML5 ви використали у розмітці сторінки та яке смислове навантаження несе кожен із них?
2. У чому полягає відмінність між блоковими та інлайновими елементами HTML?
3. Якими засобами CSS можна побудувати адаптивний макет інтерфейсу? У чому різниця між Flexbox та Grid Layout?
4. Що таке клас у JavaScript ES6+ і з яких елементів він складається (конструктор, властивості, методи)?
5. Чим відрізняється оголошення змінних через let і const? Які типи даних є примітивними в JavaScript?
6. Що таке DOM і за допомогою яких методів здійснюється пошук елементів на сторінці?
7. Які методи DOM використовуються для створення, вставлення та видалення елементів?

8. Яка різниця між властивостями innerHTML, textContent та value при роботі з елементами сторінки?

9. Як працюють події у JavaScript і чим відрізняється підхід addEventListener() від прямого призначення обробника події (onclick тощо)?

10. Що таке делегування подій, на якому механізмі воно ґрунтується та які переваги дає під час роботи з динамічно створеними елементами?

11. Як обробляється подія відправлення форми (submit) і навіщо викликається event.preventDefault()?

12. Що таке регулярний вираз, як він створюється в JavaScript і які методи (test(), match(), replace()) застосовуються для роботи з ним?

13. Які атрибути валідації форм HTML5 ви використали (required, pattern, minlength тощо) і як вони співвідносяться з валідацією засобами JavaScript?

14. Як здійснюється збереження та зчитування даних через localStorage? У якому форматі та за допомогою яких методів дані серіалізуються для збереження?

15. Які обмеження має клієнтська валідація форм і чому вона не може повністю замінити перевірку даних на стороні сервера?

2 АСИНХРОННА ВЗАЄМОДІЯ ІЗ API

2.1 Мета роботи

Метою лабораторної роботи є формування практичних навичок роботи з асинхронним кодом у JavaScript, зокрема використання конструкцій Promise, async/await, а також методів обробки помилок і виконання мережевих запитів до зовнішніх API. Здобувачі освіти повинні навчитися організовувати взаємодію між клієнтською частиною веб-додатку та сервером, здійснювати запити, обробляти відповіді та передбачати поведінку програми у разі помилок або затримок зв'язку.

Додатковою метою є ознайомлення з основами реактивного підходу до програмування на прикладі бібліотеки RxJS, а також набуття навичок підключення й використання сторонніх бібліотек (RxJS, Chart.js, Leaflet.js). У процесі виконання роботи здобувачі мають засвоїти принципи побудови гнучкого та надійного клієнтського функціоналу, здатного стабільно взаємодіяти з мережею.

2.2 Методичні вказівки з організації самостійної роботи здобувачів

Під час підготовки до лабораторної роботи здобувачі повинні самостійно опрацювати теоретичні основи асинхронного програмування у JavaScript, зокрема синтаксис і механіку роботи Promise, async/await, конструкцію try...catch, а також способи взаємодії з API через fetch. Окрему увагу варто приділити типовим помилкам мережевих запитів і принципам безпечного виконання звернень до сторонніх сервісів.

Рекомендується вивчити поняття подій у браузері та їх обробку, послідовне й паралельне виконання асинхронних задач, а також особливості роботи подієвого циклу (Event Loop) і черги мікрозадач (microtask queue). Корисно ознайомитися з прикладами побудови ланцюжків обробників, обробки затримок (setTimeout, setInterval) та моделювання сценаріїв обміну даними з API в режимі, наближеному до реального часу.

Під час самостійної роботи здобувачі повинні:

- прочитати та законспектувати базові поняття асинхронності в JavaScript;
- переглянути приклади використання fetch, Promise, async/await на прикладі запитів до публічних REST API;

- ознайомитися із синтаксисом і способом підключення бібліотек RxJS, Chart.js та Leaflet.js, які дозволяють працювати з потоками подій, візуалізацією даних і геолокаційною інформацією.
- Рекомендовані джерела для самостійного опрацювання:
- uk.javascript.info — розділ «Проміси, async/await»;
- MDN Web Docs — «Asynchronous JavaScript»;
- Eloquent JavaScript (Marijn Haverbeke) — глава 11 «Asynchronous Programming»;
- RxJS Documentation — https://rxjs.dev/;
- Chart.js Documentation — https://www.chartjs.org/docs/latest/;
- Leaflet.js Documentation — https://leafletjs.com/.

Виконання самостійної частини дасть змогу закріпити базові концепти асинхронного програмування й підготуватися до реалізації повноцінного функціоналу під час аудиторного виконання лабораторної роботи.

2.3 Опис лабораторної установки

Для виконання лабораторної роботи здобувачам освіти необхідно працювати в сучасному середовищі розробки, яке дозволяє реалізовувати та відлагоджувати JavaScript-код з використанням механізмів асинхронного програмування, обробки помилок і взаємодії з мережею через API.

Основним середовищем роботи є будь-який сучасний веб-браузер (наприклад, Google Chrome або Mozilla Firefox) із вбудованими інструментами розробника (DevTools), які дозволяють переглядати структуру сторінки, виконувати JavaScript-код, аналізувати мережеві запити у вкладці *Network* та відстежувати повідомлення про помилки у вкладці *Console*.

Для написання та редагування коду рекомендується використовувати текстовий редактор або інтегроване середовище розробки, зокрема Visual Studio Code, яке забезпечує підсвічування синтаксису, автодоповнення та підтримку корисних розширень.

У межах цієї лабораторної роботи застосовуються такі основні технології:

- **HTML та CSS** — для створення базового інтерфейсу користувача та відображення отриманих даних;

- **JavaScript (ES6+)** — для реалізації логіки асинхронних операцій через конструкції Promise, async/await та try...catch;
- **публічні тестові REST API** — для тренування навичок надсилання запитів GET/POST та аналізу структури відповідей;
- **RxJS** — підключається через CDN для реалізації реактивної обробки подій.

Для розширення функціональності додатків і демонстрації практичних кейсів можуть використовуватися допоміжні бібліотеки: RxJS, Chart.js, Leaflet.js, Axios, Toastify.js, Validator.js тощо.

Таким чином, лабораторна установка забезпечує всі необхідні умови для відпрацювання практичних навичок асинхронної взаємодії з даними, перевірки стабільності роботи застосунку у випадках помилок і розробки сучасних функціональних елементів веб-інтерфейсу.

2.4 Порядок виконання роботи і методичні вказівки з її виконання

2.4.1 Теоретична частина

Асинхронне програмування — це підхід, що дозволяє виконувати тривалі або відкладені операції (запити до сервера, завантаження файлів, очікування відповіді API) без блокування основного потоку виконання. У JavaScript цей механізм особливо важливий, оскільки мова має однопотокову модель виконання та працює в середовищі браузера, де блокування інтерфейсу неприпустиме.

Подієвий цикл (Event Loop). JavaScript керує асинхронними завданнями за допомогою подієвого циклу. Коли виконується асинхронна дія, вона передається до черги мікрозадач (microtasks) або макрозадач (macrotasks), і після завершення поточного синхронного коду ці задачі поступово обробляються. Це дозволяє браузеру реагувати на події (натискання, завантаження даних, відповіді сервера) без затримок інтерфейсу. Обробники промісів (.then, .catch, .finally) завжди потрапляють у чергу мікрозадач і виконуються після завершення поточного синхронного коду, але раніше за макрозадачі (наприклад, setTimeout).

Promise. Проміс — це об'єкт-обгортка над асинхронною дією, що може перебувати в одному з трьох станів: pending (очікування), fulfilled (виконано успішно) або rejected (відхилено з помилкою). Проміс дозволяє організовувати ланцюжки обробки результату методом .then() і обробки помилок методом .catch(). Якщо проміс відхиляється і не має жодного обробника .catch, браузер генерує подію unhandledrejection, що дозволяє відстежувати необроблені помилки.

async/await. Синтаксис, зручніший для читання, який дозволяє писати асинхронний код у вигляді послідовного. Ключове слово async перед функцією означає, що вона завжди повертає проміс; ключове слово await усередині такої

функції призупиняє її виконання до завершення промісу та повертає отриманий результат (або передає помилку далі, якщо проміс відхилено).

Обробка помилок (try...catch). Асинхронний код обов'язково потребує обробки помилок, щоб уникнути аварійного завершення програми або втрати контролю над інтерфейсом. Найпоширеніші причини помилок — відсутність відповіді від сервера, помилки формату даних, некоректні вхідні значення тощо. Конструкція try...catch дозволяє ізолювати проблемну ділянку коду (зазвичай — виклик з await) і коректно відреагувати на виняткову ситуацію, не зупиняючи роботу всієї програми.

fetch. Сучасний інтерфейс для виконання HTTP-запитів, що замінює застарілий XMLHttpRequest і працює на основі Promise. Підтримує стандартні методи GET, POST, PUT, DELETE, а також передавання заголовків і тіла запиту. Важливо пам'ятати, що fetch вважає запит «успішним» (не переходить у catch) навіть при HTTP-статусах на кшталт 404 або 500 — такі випадки потрібно перевіряти вручну через властивість response.ok або response.status. Більшість сучасних API повертають дані у форматі JSON (JavaScript Object Notation); метод response.json() дозволяє перетворити тіло відповіді на JavaScript-об'єкт.

Основи RxJS. RxJS — бібліотека для реактивного програмування, в основі якої лежить поняття потоку подій (Observable) — джерела даних, на яке можна підписатися (subscribe) і отримувати нові значення в міру їх появи: клацання мишею, введення тексту, спрацювання таймера чи мережева відповідь. На відміну від Promise, який завжди повертає рівно одне значення, Observable може випромінювати багато значень протягом часу і не починає «виконуватися», доки на нього ніхто не підписався.

Таблиця 2.1 — Найпоширеніші оператори RxJS, застосовані в цій роботі

Оператор	Призначення
fromEvent(target, 'подія')	перетворює подію DOM (клік, введення тексту) на потік Observable
debounceTime(мс)	пропускає нове значення потоку лише після паузи вказаної тривалості (зручно для пошуку «під час введення»)
switchMap(fn)	перемикається на новий внутрішній Observable (наприклад, новий запит fetch), автоматично скасовуючи попередній
map, filter	перетворення та фільтрація значень потоку
interval(мс)	створює потік значень через рівні проміжки часу (для періодичного оновлення даних)
mergeMap	паралельно обробляє кілька внутрішніх Observable, не скасовуючи попередні

Оператор	Призначення
forkJoin, combineLatest	очікують завершення або комбінують кілька Observable-потоків (аналог Promise.all для RxJS)

Підключити RxJS у навчальному проєкті найпростіше через CDN:

<script src="https://unpkg.com/rxjs@7.8.1/dist/bundles/rxjs.umd.min.js"></script>

після чого всі інструменти бібліотеки стають доступні через глобальний об'єкт rxjs (наприклад, rxjs.fromEvent, rxjs.interval).

2.4.2 Практична частина

Практична частина лабораторної роботи передбачає реалізацію веб-застосунку, що демонструє використання механізмів асинхронного програмування, обробки помилок та взаємодії з мережевими ресурсами через API. Рекомендований покроковий алгоритм виконання:

Крок 1. Розробка інтерфейсу користувача. Створити HTML-структуру сторінки, що містить необхідні елементи — поле введення, кнопки, область виведення результату (список, картки, таблиця тощо) — відповідно до вимог обраного варіанта.

Крок 2. Налаштування обробки подій через RxJS. Підключити RxJS через CDN і за допомогою fromEvent перетворити взаємодію користувача (клік, введення тексту, зміну значення в полі) на потік подій. За потреби застосувати debounceTime для відкладеного реагування на введення тексту.

Крок 3. Виконання асинхронного запиту до зовнішнього API. У функції-обробнику потоку, позначеній як async, викликати fetch з await для звернення до обраного публічного REST API. За потреби (наприклад, для одночасного виконання кількох запитів) використати Promise.all/Promise.allSettled або оператор RxJS switchMap/mergeMap.

Крок 4. Обробка відповіді та виведення результату в DOM. Перевірити успішність запиту (response.ok), перетворити тіло відповіді методом response.json() і динамічно сформувати розмітку (елементи списку, картки, рядки таблиці) для відображення отриманих даних користувачу.

Крок 5. Обробка помилок. Огорнути асинхронний виклик конструкцією try...catch, щоб коректно реагувати на відсутність відповіді сервера, мережеві збої або некоректний формат даних. Вивести користувачу зрозуміле повідомлення про помилку замість «мовчазного» збою інтерфейсу.

Крок 6. (за потреби варіанта) Підключення допоміжних бібліотек для візуалізації:

```
<!-- Chart.js — для побудови графіків -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
<!-- Leaflet.js — для роботи з інтерактивними картами -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```

Крок 7. Тестування та підготовка звіту. Перевірити стабільність взаємодії з API (зокрема реакцію застосунку на відсутність з'єднання чи помилкові відповіді сервера), відповідність реалізації вимогам обраного варіанта та коректність відображення даних, після чого оформити звіт відповідно до вимог розділу 2.6.

2.5 Завдання

Кожна бригада (до двох здобувачів) обирає один із наведених варіантів (таблиця 2.2) за списком у журналі одного зі здобувачів та розробляє відповідний веб-додаток. За бажанням можна використати інше відкрите API замість запропонованого в завданні — в межах тієї самої предметної області. Необхідно виконати основне завдання обраного варіанта та все додаткове завдання, зазначене у відповідному варіанті.

Таблиця 2.2 – Основне завдання для лабораторної роботи № 2

Варіант	Завдання
Варіант 1. Пошук GIF-анімацій (Giphy API)	**Мета:** пошук GIF-анімацій через Giphy API. **Інтерфейс:** поле пошуку; кнопка «Пошук»; сітка результатів. **Функції:** обробка подій через RxJS; асинхронний запит при пошуку; вивід GIF; обробка помилок. **Додаткове завдання:** автозаповнення з debounce 300 мс; нескінченний скролінг. **Рекомендації:** fetch+async/await; RxJS fromEvent, debounceTime, switchMap; індикатор завантаження. **Примітка:** дотримуватися лімітів Giphy API.
Варіант 2. Прогноз погоди (OpenWeatherMap API)	**Мета:** поточна погода й прогноз для міста через OpenWeatherMap API. **Інтерфейс:** поле назви міста; кнопка «Отримати прогноз»; блок погоди й прогнозу. **Функції:** RxJS для подій; асинхронний запит при натисканні; вивід даних; обробка помилок. **Додаткове завдання:** автооновлення кожні 10 хв (interval); історія 5 останніх запитів. **Рекомендації:** fetch+async/await; RxJS fromEvent, debounceTime, switchMap.

Варіант	Завдання
	Примітка: дотримуватися лімітів OpenWeatherMap API.
Варіант 3. Новинний агрегатор (NewsAPI)	**Мета:** стрічка новин з різних джерел через NewsAPI. **Інтерфейс:** поле пошуку; кнопка «Пошук»; список новин з переходом до повного тексту. **Функції:** RxJS для подій; асинхронний запит; вивід новин; обробка помилок. **Додаткове завдання:** фільтр за категорією; автооновлення стрічки кожні 5 хв (interval). **Рекомендації:** fetch+async/await; RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися лімітів NewsAPI.
Варіант 4. Пошук книг в онлайн-бібліотеці (Google Books API)	**Мета:** пошук книг за назвою/автором/ключовими словами через Google Books API. **Інтерфейс:** поле пошуку; кнопка «Пошук»; список книг (назва, автор, опис, обкладинка). **Функції:** RxJS для подій; асинхронний запит; вивід результатів; обробка помилок. **Додаткове завдання:** автозаповнення з debounce 300 мс; кілька одночасних запитів з виводом у 3 секціях (назва/автори/жанри). **Рекомендації:** реєстрація в Google Cloud Console; fetch+async/await; RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися лімітів Google Books API.
Варіант 5. Конвертер валют (ExchangeRate API)	**Мета:** конвертація сум між валютами через ExchangeRate API. **Інтерфейс:** поле суми; списки валют; поле результату. **Функції:** RxJS для подій; асинхронний запит при зміні суми/валют; конвертація в реальному часі; обробка помилок. **Додаткове завдання:** графік курсу за тиждень (Chart.js); автооновлення кожні 5 хв (interval). **Рекомендації:** реєстрація на ExchangeRate API; fetch+async/await; RxJS allSettled, fromEvent, switchMap.

Варіант	Завдання
	Примітка: дотримуватися лімітів ExchangeRate API.
Варіант 6. Відстеження цін на криптовалюти (CoinGecko API)	**Мета:** відстеження цін обраних криптовалют у реальному часі через CoinGecko API. **Інтерфейс:** список обраних криптовалют з цінами; додавання/видалення зі списку. **Функції:** RxJS для подій; асинхронні запити; оновлення в реальному часі; обробка помилок. **Додаткове завдання:** графік зміни цін (Chart.js); сповіщення (Notification) про ціновий рівень. **Рекомендації:** fetch+async/await; RxJS allSettled, fromEvent, interval. **Примітка:** дотримуватися лімітів CoinGecko API.
Варіант 7. Моніторинг авіаційних польотів (Flightradar24 API або аналог)	**Мета:** відстеження авіарейсів у реальному часі. **Інтерфейс:** інтерактивна карта (Leaflet.js); пошук за рейсом/авіакомпанією; список активних рейсів. **Функції:** RxJS для подій; асинхронні запити; вивід на карту й у список; обробка помилок. **Додаткове завдання:** деталі рейсу (маршрут, час, висота, швидкість); сповіщення про статус рейсу. **Рекомендації:** fetch+async/await; RxJS allSettled, interval. **Примітка:** дотримуватися лімітів обраного сервісу.
Варіант 8. Генератор випадкових жартів (JokeAPI)	**Мета:** отримання випадкових жартів через JokeAPI. **Інтерфейс:** поле кількості жартів N; кнопка «Згенерувати»; картки жартів зі статусом завантаження. **Функції:** RxJS для події кнопки; N незалежних запитів, кожен виводиться одразу після отримання. **Додаткове завдання:** фільтр за категоріями; автооновлення жарту кожні 30 с (interval). **Рекомендації:** fetch+async/await; RxJS allSettled, all, switchMap. **Примітка:** дотримуватися лімітів обраного сервісу.
Варіант 9. Пошук кулінарних рецептів (TheMealDB API)	**Мета:** пошук рецептів за назвою/інгредієнтом через TheMealDB API. **Інтерфейс:** поле пошуку; кнопка «Знайти рецепт»; картки страв.

Варіант	Завдання
	Функції: RxJS для подій; асинхронний запит; деталі рецепта за кліком (інгредієнти, кроки приготування); обробка помилок. **Додаткове завдання:** кнопка «Порадуй мене» (випадкова страва); фільтр за категорією/кухнею світу. **Рекомендації:** fetch+async/await; RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися лімітів безкоштовного рівня TheMealDB.
Варіант 10. Каталог фільмів та серіалів (TMDb API)	**Мета:** пошук фільмів/серіалів за назвою через TMDb API. **Інтерфейс:** поле пошуку; кнопка «Пошук»; сітка карток (постер, назва, рік, рейтинг). **Функції:** RxJS для подій; асинхронний запит; вивід результатів; обробка помилок. **Бонус:** автозаповнення з debounce 300 мс; детальна сторінка фільму (сюжет, жанри, актори) за кліком. **Рекомендації:** реєстрація й API-ключ TMDb; fetch+async/await; RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися лімітів TMDb API.
Варіант 11. Астрономічне фото дня (NASA APOD API)	**Мета:** перегляд астрофото дня та архівних знімків NASA. **Інтерфейс:** вибір дати або кнопка «Сьогодні»; кнопка «Показати»; зображення/відео із заголовком і описом. **Функції:** RxJS для подій; асинхронний запит при зміні дати; обробка помилок (відсутність даних за датою). **Додаткове завдання:** перегляд діапазону дат галереєю (Promise.allSettled); кнопка «Випадковий день в історії космосу». **Рекомендації:** демо-ключ на api.nasa.gov; fetch+async/await; RxJS switchMap, forkJoin. **Примітка:** дотримуватися лімітів демо-ключа NASA API.
Варіант 12. Енциклопедія Pokémon (PokéAPI)	**Мета:** довідник покемонів за назвою/номером через PokéAPI.

Варіант	Завдання
	Інтерфейс: поле пошуку; кнопка «Знайти»; картка із зображенням, типами, характеристиками. **Функції:** RxJS для подій; асинхронний запит; обробка помилок (покемона не знайдено). **Додаткове завдання:** порівняння двох покемонів паралельно (Promise.all, діаграма Chart.js); ланцюжок еволюції із зображеннями. **Рекомендації:** fetch+async/await (без ключа); RxJS fromEvent, debounceTime, forkJoin. **Примітка:** дотримуватися рекомендованої частоти запитів PokéAPI.
Варіант 13. Довідник країн світу (REST Countries API)	**Мета:** перегляд і пошук інформації про країни (столиця, населення, регіон, прапор, мови). **Інтерфейс:** поле пошуку країни; фільтр за регіоном; картки з прапором і даними. **Функції:** RxJS для подій; асинхронний запит при вводі/зміні фільтра; обробка помилок. **Додаткове завдання:** сортування за назвою/населенням; сусідні країни паралельними запитами (Promise.allSettled). **Рекомендації:** fetch+async/await (без ключа); RxJS fromEvent, debounceTime, switchMap. **Примітка:** звертати увагу на актуальну версію ендпоінтів API.
Варіант 14. Трекер космічних запусків (Launch Library API, The Space Devs)	**Мета:** перегляд найближчих запланованих космічних запусків. **Інтерфейс:** список запусків (місія, ракета, космодром, дата); фільтр за агентством; зворотний відлік. **Функції:** RxJS для подій і відліку; асинхронний запит; обробка помилок. **Додаткове завдання:** автооновлення статусу кожні 5 хв (interval); деталі місії за кліком. **Рекомендації:** fetch+async/await; RxJS interval, fromEvent, switchMap. **Примітка:** дотримуватися лімітів безкоштовного рівня API.

Варіант	Завдання
Варіант 15. Електронний словник англійської мови (Free Dictionary API)	**Мета:** визначення, транскрипція та приклади вживання англійських слів. **Інтерфейс:** поле слова; кнопка «Знайти»; транскрипція, визначення, синоніми, кнопка вимови. **Функції:** RxJS для подій; асинхронний запит; обробка помилок (слово не знайдено). **Додаткове завдання:** історія 5 слів з debounce-пошуком; пошук кількох слів одночасно (Promise.allSettled). **Рекомендації:** fetch+async/await (без ключа); RxJS fromEvent, debounceTime, mergeMap. **Примітка:** дотримуватися лімітів Free Dictionary API.
Варіант 16. Онлайн-перекладач тексту (LibreTranslate API)	**Мета:** переклад введеного тексту між мовами через LibreTranslate API. **Інтерфейс:** поле тексту; вибір вихідної/цільової мови; кнопка «Перекласти»; поле результату. **Функції:** RxJS для подій; асинхронний запит; переклад у реальному часі; обробка помилок. **Додаткове завдання:** автопереклад під час набору тексту (debounce 500 мс) без кнопки; кнопка «Поміняти мови місцями» з повторним перекладом. **Рекомендації:** fetch+async/await; RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися лімітів символів/запитів обраного сервісу перекладу.
Варіант 17. Генератор QR-кодів (QR Server API)	**Мета:** генерація QR-коду за введеним текстом/URL через QR Server API (goqr.me) або аналог. **Інтерфейс:** поле тексту/URL; вибір розміру зображення; кнопка «Згенерувати»; область із QR-кодом і завантаженням. **Функції:** RxJS для подій; асинхронний запит; вивід зображення; обробка помилок (порожній текст/перевищення довжини). **Додаткове завдання:** миттєва генерація під час набору тексту (debounce 400 мс); історія 5 останніх згенерованих кодів.

Варіант	Завдання
	Рекомендації: fetch+async/await; RxJS fromEvent, debounceTime. **Примітка:** дотримуватися лімітів обраного сервісу.
Варіант 18. Визначення геолокації за IP-адресою (ipapi.co)	**Мета:** визначення країни/міста/часового поясу за IP-адресою через ipapi.co. **Інтерфейс:** поле IP-адреси (необов 'язкове); кнопка «Визначити»; дані локації + позначка на карті (Leaflet.js). **Функції:** RxJS для події кнопки; асинхронний запит; вивід на карту; обробка помилок (некоректний IP). **Додаткове завдання:** автовизначення власної геолокації при завантаженні сторінки; місцевий час з оновленням щосекунди (interval). **Рекомендації:** fetch+async/await (без ключа для базового рівня); RxJS fromEvent, interval. **Примітка:** дотримуватися лімітів безкоштовних запитів на добу.
Варіант 19. Каталог аніме та манги (Jikan API)	**Мета:** пошук аніме/манги за назвою через Jikan API (неофіційний API MyAnimeList). **Інтерфейс:** поле пошуку; кнопка «Пошук»; картки (обкладинка, назва, рейтинг, кількість епізодів). **Функції:** RxJS для подій; асинхронний запит; вивід результатів; обробка помилок. **Додаткове завдання:** автозаповнення з debounce 400 мс; детальна сторінка за кліком (опис сюжету, жанри, схожі тайтли). **Рекомендації:** fetch+async/await (без ключа); RxJS fromEvent, debounceTime, switchMap. **Примітка:** дотримуватися рекомендованого rate limit Jikan API.
Варіант 20. Календар державних свят (Nager.Date API)	**Мета:** список державних свят обраної країни за обраний рік через Nager.Date API. **Інтерфейс:** вибір країни; вибір року; таблиця/список свят у хронологічному порядку. **Функції:** RxJS для подій зміни параметрів; асинхронний запит; обробка помилок (немає даних). **Додаткове завдання:** виділення найближчого свята з лічильником днів (interval); порівняння кількості

Варіант	Завдання
	вихідних у двох країнах паралельними запитами (Promise.all). **Рекомендації:** fetch+async/await (без ключа); RxJS fromEvent, switchMap, interval, forkJoin. **Примітка:** дотримуватися умов використання Nager.Date API.

2.6 Зміст звіту

За результатами виконання лабораторної роботи здобувачі повинні підготувати звіт, який містить повну інформацію про хід виконання завдання та отримані результати. Звіт має бути структурованим і складатися з кількох обов'язкових розділів.

Першим елементом є титульна сторінка, на якій зазначаються повна назва навчального закладу, назва дисципліни, назва лабораторної роботи, прізвище та ім'я здобувача, а також номер академічної групи.

Першим розділом звіту є мета роботи, де коротко формулюється основна ціль лабораторної роботи та завдання, які потрібно було реалізувати під час її виконання.

У розділі «Хід роботи» необхідно вказати, який саме варіант завдання було обрано здобувачем або бригадою. Далі слід подати код програми з поясненням основних фрагментів — це дозволяє продемонструвати логіку реалізації та розуміння принципів роботи. Також потрібно включити результати роботи програми у вигляді скріншотів інтерфейсу користувача (UI) з даними, отриманими від обраного API.

Останнім є розділ висновків, у якому здобувач повинен узагальнити отримані результати, оцінити ефективність виконаної роботи, зазначити труднощі, з якими він зіткнувся, та можливі шляхи вдосконалення реалізованого функціоналу.

2.7 Контрольні запитання і завдання

1. Що таке асинхронне програмування і навіщо воно використовується у веб-розробці?
2. Поясніть механізм роботи подієвого циклу (Event Loop) у JavaScript.
3. Що таке Promise і які стани він може мати?
4. Яка різниця між методами .then().catch() та async/await?
5. Як обробляються помилки у коді з async/await?

6. Наведіть приклад запиту до API за допомогою fetch, що обробляє відповідь у форматі JSON.

7. Що таке try...catch? У яких випадках він використовується?

8. Як перевірити, що HTTP-запит повернув помилку (наприклад, статус 404 або 500)?

9. Яка структура відповіді у форматі JSON і як з нею працювати в JavaScript?

10. Що таке REST API? Назвіть основні HTTP-методи, які використовуються при взаємодії з API.

3 СТВОРЕННЯ КОМПОНЕНТІВ НА REACT

3.1 Мета роботи

Метою лабораторної роботи є ознайомлення здобувачів освіти з декларативним підходом до побудови користувацьких інтерфейсів на основі бібліотеки React. Здобувачі повинні набути практичних навичок створення багаторазових функціональних компонентів, розподілу інтерфейсу на незалежні структурні одиниці (декомпозиції), управління локальним станом компонента за допомогою хука useState, а також налаштування маршрутизації в односторінковому застосунку (SPA) засобами бібліотеки React Router.

3.2 Методичні вказівки з організації самостійної роботи здобувачів

Під час підготовки до лабораторної роботи здобувачі повинні самостійно опрацювати теоретичні основи бібліотеки React, зокрема принцип роботи Virtual DOM та його роль у продуктивності оновлення інтерфейсу, синтаксис JSX, поняття функціонального компонента та різницю між props і state. Особливу увагу варто приділити основним хукам — useState (для управління локальним станом) та useEffect (для роботи з побічними ефектами) — а також принципам їх коректного використання.

Окремим напрямом самостійної підготовки є вивчення бібліотеки React Router: призначення компонентів BrowserRouter, Routes, Route, механізм навігації через Link та Navigate, а також отримання параметрів динамічного маршруту за допомогою useParams.

За бажанням здобувачі можуть додатково ознайомитися з UI-бібліотеками компонентів — Material UI або Bootstrap (react-bootstrap) — які дозволяють пришвидшити стилізацію інтерфейсу готовими компонентами (кнопки, картки, форми, модальні вікна тощо), проте їх використання не є обов'язковим.

Під час самостійної роботи здобувачі повинні:

- прочитати та законспектувати основні поняття компонентного підходу, Virtual DOM, JSX, props і state;
- переглянути офіційну документацію React щодо хуків useState та useEffect;
- ознайомитися з документацією React Router щодо базового налаштування маршрутів;

- встановити розширення React Developer Tools для браузера та ознайомитися з його можливостями.
- Рекомендовані джерела для самостійного опрацювання:
- офіційна документація React — https://react.dev/;
- офіційна документація React Router — https://reactrouter.com/;
- MDN Web Docs — «Introduction to client-side frameworks»;
- документація Material UI — https://mui.com/ або React-Bootstrap — https://react-bootstrap.github.io/.

Виконання самостійної частини дасть змогу здобувачам впевнено орієнтуватися в компонентній структурі React-застосунку та підготуватися до практичної реалізації повноцінного SPA під час аудиторного заняття.

3.3 Опис лабораторної установки

Для виконання лабораторної роботи здобувачам необхідне таке програмне забезпечення:

- **Node.js** (актуальна LTS-версія) — середовище виконання JavaScript поза браузером, необхідне для роботи інструментів збирання React-проєкту та менеджера пакетів;
- **npm** (постачається разом із Node.js) — для встановлення залежностей проєкту, зокрема бібліотеки react-router-dom;
- **сучасний веб-браузер** (Google Chrome або Mozilla Firefox) із встановленим розширенням **React Developer Tools**, яке дозволяє переглядати дерево компонентів, значення props і state кожного компонента в реальному часі;
- **редактор коду Visual Studio Code** з рекомендованими розширеннями для JavaScript/React (підсвічування синтаксису JSX, ESLint, Prettier).

Якщо Node.js ще не встановлено, необхідно завантажити рекомендовану LTS-версію з офіційного сайту https://nodejs.org/ та встановити її, дотримуючись інструкцій інсталятора.

Лабораторне середовище дозволяє ініціалізувати новий React-проєкт (за допомогою create-react-app або сучаснішого інструмента Vite), розробляти компонентну структуру застосунку, налагоджувати роботу props і state через React

Developer Tools, а також перевіряти коректність маршрутизації безпосередньо в адресному рядку браузера.

3.4 Порядок виконання роботи і методичні вказівки з її виконання

3.4.1 Теоретична частина

React та декларативний підхід. На відміну від імперативного JavaScript, де розробник власноруч описує послідовність дій для зміни кожного елемента сторінки (document.getElementById(...), element.classList.add(...) тощо), React пропонує декларативний підхід: розробник лише описує, **яким повинен виглядати інтерфейс** для певного стану даних, а сама бібліотека визначає, які саме зміни необхідно внести в реальний DOM.

Virtual DOM. В основі механізму оновлення інтерфейсу React лежить віртуальний DOM — легка JavaScript-копія реального дерева DOM. Під час зміни стану компонента React спочатку створює нове віртуальне дерево, порівнює його з попереднім (процес «diffing») і застосовує до реального DOM лише мінімально необхідний набір змін. Це суттєво підвищує продуктивність порівняно з прямою маніпуляцією DOM, особливо у великих застосунках.

Функціональні компоненти та JSX. Функціональний компонент — це звичайна JavaScript-функція, яка повертає розмітку у форматі JSX (розширення синтаксису JavaScript, що дозволяє описувати структуру інтерфейсу в HTML-подібному вигляді безпосередньо в коді). Використання функціональних компонентів у поєднанні з хуками є сучасним стандартом розробки на React.

Props і State. *Props* (скорочення від *properties*) — дані, які батьківський компонент передає дочірньому через атрибути; props є незмінними (immutable) всередині дочірнього компонента й оновлюються лише через повторну передачу з батьківського компонента. *State* — внутрішній стан компонента, призначений для зберігання даних, що змінюються протягом «життя» компонента (введення в полі форми, перелік елементів колекції тощо). Головне правило: props використовуються для даних, що надходять «ззовні» й не змінюються самим компонентом, тоді як state — для даних, керованих самим компонентом.

Основні хуки.

- useState(initialValue) — повертає масив із двох елементів: поточне значення стану та функцію для його оновлення. Виклик функції оновлення ініціює повторний рендеринг компонента з новим значенням.
- useEffect(callback, [deps]) — дозволяє виконувати «побічні ефекти» (запити, підписки, робота з таймерами тощо) на певних етапах життєвого циклу компонента: без масиву залежностей — після кожного рендеру; з порожнім

масивом [] — лише один раз після першого монтажу; із заданими залежностями — щоразу, коли ці значення змінюються.

React Router. Для реалізації маршрутизації в SPA-застосунку використовується бібліотека react-router-dom. Основні компоненти:

- BrowserRouter — кореневий компонент, що обгортає весь застосунок і вмикає підтримку маршрутизації через History API браузера;
- Routes — контейнер, що об'єднує набір можливих маршрутів застосунку;
- Route — окремий маршрут, що визначає, який компонент рендерити за певним шляхом (path), у тому числі динамічним (наприклад, /books/:id);
- Link — компонент для створення навігаційних посилань без перезавантаження сторінки (замість звичайного <a href="…">);
- Navigate / useNavigate — засоби програмного перенаправлення на інший маршрут (наприклад, після успішного збереження форми);
- useParams — хук для отримання значення динамічного параметра маршруту (наприклад, id обраного елемента) усередині компонента сторінки.

3.4.2 Практична частина

Рекомендований покроковий алгоритм виконання лабораторної роботи:

Крок 1. Ініціалізація проєкту. Створити новий React-проєкт за допомогою npx create-react-app my-app або, як сучаснішу та швидшу альтернативу, npm create vite@latest my-app -- --template react. Встановити бібліотеку маршрутизації командою npm install react-router-dom.

Крок 2. Створення структури тек проєкту. Організувати вихідний код за файловою структурою, що розділяє відповідальність: тека components/ — для багаторазових дрібних UI-компонентів (картки, елементи списку, кнопки), тека pages/ — для компонентів-сторінок, що відповідають конкретному маршруту (наприклад, HomePage.jsx, DetailsPage.jsx), тека routes/ (за потреби) — для конфігурації маршрутизації, якщо вона виноситься в окремий файл.

Крок 3. Декомпозиція інтерфейсу на компоненти. Виділити з макета інтерфейсу незалежні багаторазові компоненти — типово: Header (шапка з навігацією), Footer (підвал сторінки), компонент картки/елемента списку (наприклад, BookCard, TaskItem — залежно від обраного варіанта) та компонент форми (…Form) для додавання чи редагування запису. Кожен компонент повинен відповідати за одну чітко визначену частину інтерфейсу.

Крок 4. Налаштування маршрутизації. Обгорнути кореневий компонент застосунку в <BrowserRouter>, визначити щонайменше 2–3 маршрути за допомогою <Routes> і <Route>: сторінку списку елементів (шлях /), сторінку

деталей або редагування обраного елемента (шлях виду /entity/:id) та, за потреби, окрему сторінку форми додавання нового запису (/entity/new). Реалізувати навігацію між сторінками за допомогою компонента Link у списку та в шапці сайту.

Крок 5. Робота зі станом та передача даних між компонентами. Оголосити основний масив даних (колекцію об'єктів обраного варіанта) у локальному стані компонента-контейнера (найчастіше — компонента сторінки списку або кореневого компонента App) за допомогою useState. Передати цей масив та функції для його зміни (додавання, редагування) дочірнім компонентам через props. Для сторінки деталей/редагування — знайти потрібний елемент масиву за параметром маршруту, отриманим через useParams.

Крок 6. Стилізація інтерфейсу. Оформити зовнішній вигляд компонентів за допомогою звичайного CSS, CSS-модулів або, за бажанням, готової UI-бібліотеки (Material UI, React-Bootstrap), розглянутої під час самостійної підготовки.

Крок 7. Тестування та підготовка звіту. Перевірити коректність переходів між усіма реалізованими маршрутами, коректність відображення й оновлення даних після додавання/редагування запису, а також відсутність помилок у консолі браузера. За допомогою React Developer Tools переконатися, що дерево компонентів відповідає задуманій декомпозиції, після чого оформити звіт відповідно до вимог розділу 3.6.

3.5 Завдання

Кожна бригада (до двох здобувачів) або здобувач індивідуально продовжує роботу над предметною областю свого варіанта, обраного під час виконання лабораторної роботи № 1 (таблиця 3.1 містить той самий перелік предметних областей із тією самою нумерацією, що й таблиця 1.1). Номер варіанта не змінюється: застосунок, раніше реалізований засобами нативного JavaScript, переробляється як SPA-застосунок засобами React.

Незалежно від обраного варіанта, реалізований застосунок повинен відповідати таким загальним вимогам:

- модель даних (перелік полів основної сутності) та базовий набір операцій відповідають опису варіанта з лабораторної роботи № 1; допускається розширення моделі новими полями, але не заміна предметної області;
- проєкт створено на основі Create React App або Vite з використанням **виключно функціональних компонентів**;
- інтерфейс декомпоновано щонайменше на 4–5 компонентів (наприклад, Header, Footer, компонент картки/елемента списку, компонент детального перегляду, компонент форми);

- налаштовано маршрутизацію за допомогою React Router: мінімум дві сторінки — сторінка списку елементів (/), сторінка деталей та редагування окремого елемента (/entity/:id); за потреби додається окрема сторінка форми додавання нового запису (/entity/new);
- колекція даних (масив об'єктів) зберігається в локальному стані (useState) компонента-контейнера та передається дочірнім компонентам через props;
- реалізовано щонайменше додавання нового запису та перегляд/редагування/видалення наявного запису;
- **використання Redux або будь-якого іншого стороннього менеджера глобального стану на цьому етапі не допускається** — усі дані застосунку зберігаються виключно в локальному стані компонентів.

Таблиця 3.1 – Основне завдання для лабораторної роботи № 3

Варіант та назва	Завдання
Варіант 1. Система управління бібліотекою	Сторінка `/` — список книг (назва, автор, рік видання, жанр) з фільтрацією за жанром; сторінка `/books/:id` — детальна інформація про книгу та її редагування. **Основні компоненти:** `Header`, `BookList`, `BookCard`, `BookDetails`, `BookForm`. **Приклад запису:** `{ id, title, author, year, genre, rating }`
Варіант 2. Менеджер завдань (To-Do List)	Сторінка `/` — список завдань зі статусом виконання; сторінка `/tasks/:id` — деталі та редагування обраного завдання. **Основні компоненти:** `Header`, `TaskList`, `TaskItem`, `TaskDetails`, `TaskForm`. **Приклад запису:** `{ id, title, description, status, deadline }`
Варіант 3. Система обліку студентів	Сторінка `/` — список студентів із середнім балом; сторінка `/students/:id` — картка студента з переліком оцінок. **Основні компоненти:** `Header`, `StudentList`, `StudentCard`, `StudentDetails`, `StudentForm`.

Варіант та назва	Завдання
	Приклад запису: `{ id, name, grades, averageScore }`
Варіант 4. Інтерактивний словник	Сторінка `/` — список слів; сторінка `/words/:id` — визначення слова та приклад вживання. **Основні компоненти:** `Header`, `WordList`, `WordItem`, `WordDetails`, `WordForm`. **Приклад запису:** `{ id, word, definition, example }`
Варіант 5. Калькулятор витрат	Сторінка `/` — список витрат та загальна сума за обраний період; сторінка `/expenses/:id` — деталі й редагування запису. **Основні компоненти:** `Header`, `ExpenseList`, `ExpenseItem`, `ExpenseDetails`, `ExpenseForm`, `Summary`. **Приклад запису:** `{ id, amount, category, date }`
Варіант 6. Аналізатор тексту	Сторінка `/` — форма аналізу тексту та історія попередніх аналізів; сторінка `/history/:id` — деталі обраного запису історії. **Основні компоненти:** `Header`, `TextAnalyzerForm`, `ResultSummary`, `HistoryList`, `HistoryDetails`. **Приклад запису:** `{ id, text, wordCount, charCount, date }`
Варіант 7. Галерея зображень	Сторінка `/` — сітка мініатюр зображень; сторінка `/images/:id` — перегляд зображення в збільшеному розмірі з описом. **Основні компоненти:** `Header`, `Gallery`, `ImageCard`, `ImageDetails`, `ImageForm`. **Приклад запису:** `{ id, title, url, description }`
Варіант 8. Калькулятор площі фігур	Сторінка `/` — форма вибору фігури, параметрів та результат обчислення; сторінка `/history/:id` — деталі попереднього розрахунку.

Варіант та назва	Завдання
	Основні компоненти: `Header`, `ShapeForm`, `ResultCard`, `HistoryList`, `HistoryDetails`. **Приклад запису:** `{ id, shapeType, params, area }`
Варіант 9. Система тестування	Сторінка `/` — список доступних тестів; сторінка `/tests/:id` — проходження тесту з питаннями та підрахунком результату. **Основні компоненти:** `Header`, `TestList`, `TestRunner`, `QuestionCard`, `ResultPage`. **Приклад запису:** `{ id, question, options, correctAnswerIndex }`
Варіант 10. Система управління контактами	Сторінка `/` — список контактів, згрупованих за категорією; сторінка `/contacts/:id` — деталі та редагування контакту. **Основні компоненти:** `Header`, `ContactList`, `ContactCard`, `ContactDetails`, `ContactForm`. **Приклад запису:** `{ id, name, phone, email, group }`
Варіант 11. Каталог фільмів та серіалів	Сторінка `/` — список фільмів з фільтрацією за жанром; сторінка `/movies/:id` — деталі фільму з рейтингом і статусом перегляду. **Основні компоненти:** `Header`, `MovieList`, `MovieCard`, `MovieDetails`, `MovieForm`. **Приклад запису:** `{ id, title, genre, year, rating, watched }`
Варіант 12. Трекер щоденних звичок	Сторінка `/` — список звичок з поточною серією виконання (streak); сторінка `/habits/:id` — детальна статистика звички. **Основні компоненти:** `Header`, `HabitList`, `HabitCard`, `HabitDetails`, `HabitForm`. **Приклад запису:** `{ id, name, completedDates, streak }`

Варіант та назва	Завдання
Варіант 13. Система бронювання столиків у ресторані	Сторінка `/` — список поточних бронювань; сторінка `/reservations/:id` — деталі та редагування бронювання з перевіркою зайнятості столика. **Основні компоненти:** `Header`, `ReservationList`, `ReservationCard`, `ReservationForm`. **Приклад запису:** `{ id, clientName, date, time, guests, tableNumber }`
Варіант 14. Міні-платформа для публікації статей (блог)	Сторінка `/` — стрічка статей; сторінка `/articles/:id` — повний текст обраної статті. **Основні компоненти:** `Header`, `ArticleList`, `ArticleCard`, `ArticleDetails`, `ArticleForm`. **Приклад запису:** `{ id, title, content, tags, publishedAt }`
Варіант 15. Щоденник фітнес-тренувань	Сторінка `/` — історія тренувань; сторінка `/workouts/:id` — деталі та редагування тренування. **Основні компоненти:** `Header`, `WorkoutList`, `WorkoutCard`, `WorkoutDetails`, `WorkoutForm`. **Приклад запису:** `{ id, exerciseType, sets, reps, weight, date }`
Варіант 16. Каталог рецептів	Сторінка `/` — список рецептів з фільтрацією за категорією; сторінка `/recipes/:id` — детальний перегляд рецепта (інгредієнти, час приготування). **Основні компоненти:** `Header`, `RecipeList`, `RecipeCard`, `RecipeDetails`, `RecipeForm`. **Приклад запису:** `{ id, title, ingredients, category, cookTime }`
Варіант 17. Планувальник подій	Сторінка `/` — список подій, відсортованих за датою; сторінка `/events/:id` — деталі та редагування події. **Основні компоненти:** `Header`, `EventList`, `EventCard`, `EventDetails`, `EventForm`. **Приклад запису:** `{ id, title, date, time, location, type }`

Варіант та назва	Завдання
Варіант 18. Трекер вакансій (пошук роботи)	Сторінка `/` — список поданих заявок зі статусом розгляду; сторінка `/applications/:id` — деталі заявки та зміна статусу. **Основні компоненти:** `Header`, `ApplicationList`, `ApplicationCard`, `ApplicationDetails`, `ApplicationForm`. **Приклад запису:** `{ id, company, position, appliedDate, status }`
Варіант 19. Облік клієнтів та тварин ветеринарної клініки	Сторінка `/` — список карток тварин-пацієнтів; сторінка `/pets/:id` — картка тварини з датами вакцинації та редагуванням. **Основні компоненти:** `Header`, `PetList`, `PetCard`, `PetDetails`, `PetForm`. **Приклад запису:** `{ id, petName, species, ownerName, lastVaccination, nextVaccination }`
Варіант 20. Персональний нотатник (щоденник)	Сторінка `/` — стрічка нотаток з фільтрацією за тегом; сторінка `/notes/:id` — повний текст нотатки та редагування. **Основні компоненти:** `Header`, `NoteList`, `NoteCard`, `NoteDetails`, `NoteForm`. **Приклад запису:** `{ id, title, text, tags, createdAt }`

Здобувач, який за погодженням із викладачем реалізував власну предметну область у лабораторній роботі № 1, продовжує роботу над нею за умови збереження всіх загальних вимог, наведених вище. Пропонувати нову предметну область на цьому етапі не допускається.

Додаткове завдання виконується здобувачем за бажанням, понад основний обсяг роботи, і оцінюється додатковими балами на високу оцінку. Можна обрати один або кілька варіантів із наведених нижче.

Таблиця 3.2 – Додаткове завдання для лабораторної роботи № 3

Варіант та назва	Завдання
Варіант 1. Custom Hook для роботи з localStorage	Створити власний хук `useLocalStorage(key, initialValue)`, що інкапсулює логіку читання та запису даних у `localStorage`, повертаючи пару значень `[value, setValue]` за аналогією з `useState`. Застосувати для збереження основної колекції даних застосунку між перезавантаженнями сторінки.
Варіант 2. Перемикання теми оформлення через Context API	Реалізувати `ThemeContext` за допомогою `createContext` і хука `useContext` для передачі поточної теми оформлення (світла/темна) та функції її перемикання без «прокидання» props через кожен рівень дерева компонентів. Додати елемент керування темою в `Header`.
Варіант 3. Валідація форми через Formik або React Hook Form	Переробити форму додавання/редагування запису з використанням бібліотеки Formik або React Hook Form: перевірка обов'язкових полів та коректності формату введених даних (число, дата, email тощо) з виведенням повідомлень про помилки під полями форми.
Варіант 4. Динамічна пагінація списку	Додати на сторінку списку пагінацію: розподіл даних на сторінки фіксованого розміру, керування поточною сторінкою через `useState`, відображення номера сторінки в URL-запиті (`?page=2`) за допомогою хука `useSearchParams`.
Варіант 5. Збереження параметрів фільтрації та сортування в URL	Реалізувати фільтрацію та/або сортування списку так, щоб обрані значення зберігалися в рядку URL-запиту (`useSearchParams`) — це дозволяє поділитися посиланням із застосованими фільтрами або оновити сторінку без втрати налаштувань.
Варіант 6. Відкладене завантаження	Реалізувати розбиття коду за маршрутами (code splitting) за допомогою `React.lazy()` та компонента `<Suspense>`: кожна сторінка-маршрут завантажується

Варіант та назва	Завдання
сторінок (Lazy Loading)	окремим JS-чанком лише в момент переходу на відповідний маршрут.
Варіант 7. Custom Hook для пошуку з debounce	Створити власний хук `useDebounce(value, delay)`, що повертає «відкладене» значення, яке оновлюється лише після паузи заданої тривалості з моменту останньої зміни введеного тексту. Застосувати для поля пошуку/фільтрації на сторінці списку.
Варіант 8. Скасування видалення (Undo) через тимчасове повідомлення	Реалізувати «м'яке» видалення: елемент одразу зникає зі списку, але протягом кількох секунд показується повідомлення з кнопкою «Скасувати», яке відновлює запис за допомогою `setTimeout` і локального стану.
Варіант 9. Перетягування для зміни порядку елементів (Drag & Drop)	Реалізувати зміну порядку елементів списку методом перетягування (нативний HTML5 Drag & Drop API або бібліотека на кшталт `@dnd-kit/core`) з відповідним оновленням порядку в локальному стані.
Варіант 10. Сторінка 404 та обробка помилок рендерингу (Error Boundary)	Додати маршрут-заглушку (`path="*"`) з компонентом `NotFoundPage` для неіснуючих шляхів. Реалізувати клас-компонент `ErrorBoundary`, що перехоплює помилки рендерингу дочірніх компонентів і показує повідомлення замість «білого екрана».

3.6 Зміст звіту

За результатами виконання лабораторної роботи здобувачі повинні підготувати звіт, що містить повну інформацію про хід виконання завдання та отримані результати.

Першим елементом звіту є титульна сторінка, на якій зазначаються повна назва навчального закладу, назва дисципліни, назва лабораторної роботи, прізвище та ім'я здобувача, а також номер академічної групи.

Далі подається розділ «Мета роботи», де стисло формулюється основна ціль лабораторної роботи та вказується обраний варіант завдання.

У розділі «Хід виконання роботи» необхідно навести схему декомпозиції інтерфейсу на компоненти (наприклад, у вигляді дерева або переліку з коротким описом призначення кожного компонента) та лістинг **лише основних компонентів** застосунку (компонент сторінки списку, сторінки деталей/редагування, ключовий дочірній компонент картки/елемента) з поясненням використання props, state та маршрутизації — наводити повний вихідний код усього проєкту не потрібно. Обов'язковою частиною розділу є скріншоти інтерфейсу користувача для кожної реалізованої сторінки/маршруту, зроблені з **відкритою панеллю React Developer Tools**, на яких видно дерево компонентів і поточні значення їхніх props/state.

Завершальним розділом є висновки, у яких здобувач узагальнює отримані результати, оцінює відповідність реалізації поставленим вимогам, описує труднощі, з якими зіткнувся під час декомпозиції інтерфейсу чи налаштування маршрутизації, а також можливі шляхи вдосконалення реалізованого застосунку (зокрема за рахунок виконаного додаткового завдання).

3.7 Контрольні запитання і завдання

1. Що таке Virtual DOM і як його використання впливає на продуктивність оновлення інтерфейсу в React?
2. У чому полягає принципова різниця між props і state компонента?
3. Чому props вважаються незмінними (immutable) всередині дочірнього компонента, і як тоді ініціюється оновлення інтерфейсу при зміні даних?
4. Що таке JSX і чим він відрізняється від звичайного HTML-коду?
5. Яких основних правил слід дотримуватися під час використання хуків у React (де і як їх можна викликати)?
6. Що повертає хук useState і як відбувається оновлення стану компонента?
7. Яку роль відіграє масив залежностей у хуку useEffect та що відбувається, якщо передати порожній масив []?
8. Навіщо в React використовуються ключі (key) під час рендерингу списків елементів і чому не рекомендується використовувати індекс масиву як ключ?
9. Яке призначення мають компоненти BrowserRouter, Routes та Route бібліотеки React Router?
10. Як реалізувати програмну навігацію між сторінками (наприклад, після успішного збереження форми) та як отримати значення параметра з динамічного маршруту (наприклад, id елемента)?

4 УПРАВЛІННЯ СТАНОМ ЧЕРЕЗ REDUX

4.1 Мета роботи

Метою лабораторної роботи є ознайомлення здобувачів освіти з централізованим підходом до управління станом у React-застосунках на основі архітектури Flux та бібліотеки Redux. Здобувачі повинні набути практичних навичок налаштування глобального сховища (Store) за допомогою Redux Toolkit, створення слайсів стану (createSlice), а також організації асинхронної взаємодії з REST API через createAsyncThunk із коректною обробкою станів завантаження (pending), успішного виконання (fulfilled) та помилки (rejected).

4.2 Методичні вказівки з організації самостійної роботи здобувачів

Під час підготовки до лабораторної роботи здобувачі повинні самостійно опрацювати основні теоретичні засади бібліотеки Redux та розглянути, чому централізоване управління станом стає доцільним у міру зростання складності React-застосунку — зокрема, проблему «prop drilling» (передавання пропсів через кілька проміжних рівнів компонентів) та складнощі синхронізації стану між непов'язаними гілками дерева компонентів.

Основою для розуміння Redux є три фундаментальні принципи:

- **єдине джерело правди** — весь стан застосунку зберігається в одному об'єкті Store;
- **стан лише для читання** — єдиний спосіб змінити стан — надіслати дію (action);
- **зміни виконуються через чисті функції** — reducer приймає попередній стан і дію та повертає новий стан, не змінюючи вхідні дані та не викликаючи побічних ефектів.

Здобувачі повинні чітко розрізняти поняття **action** (об'єкт із обов'язковим полем type та опціональним payload, що описує подію) та **reducer** (чисту функцію, яка на основі поточного стану й отриманої дії обчислює новий стан). Окремої уваги потребує ознайомлення з поняттям **middleware** — проміжного шару обробки дій між їх відправленням (dispatch) і надходженням до reducer'a; зокрема слід розглянути **Thunk-middleware**, який дозволяє диспетчеризувати не лише прості об'єкти-дії, а й функції, що виконують асинхронну логіку (наприклад, HTTP-запити) перед відправленням «справжньої» дії з результатом.

Під час самостійної роботи здобувачі повинні:

- прочитати та законспектувати три принципи Redux, поняття Store, Action, Reducer, State;
- ознайомитися з призначенням Middleware та принципом роботи Thunk;
- переглянути офіційну документацію Redux Toolkit щодо функцій configureStore, createSlice та createAsyncThunk;
- встановити розширення Redux DevTools для браузера та ознайомитися з його інтерфейсом.

Рекомендовані джерела для самостійного опрацювання:

- офіційна документація Redux — https://redux.js.org/;
- офіційна документація Redux Toolkit — https://redux-toolkit.js.org/;
- офіційна документація React-Redux — https://react-redux.js.org/;
- розділ «State Management» на react.dev — https://react.dev/learn/scaling-up-with-reducer-and-context.

4.3 Опис лабораторної установки

Для виконання лабораторної роботи здобувачам необхідне таке програмне забезпечення:

- **Node.js** (актуальна LTS-версія) — середовище виконання, необхідне для роботи інструментів збирання React-проєкту, менеджера пакетів npm та локального мок-сервера API;
- **npm** — для встановлення пакетів @reduxjs/toolkit, react-redux та (за потреби) json-server;
- **сучасний веб-браузер** (Google Chrome або Mozilla Firefox) із встановленими розширеннями: **React Developer Tools** (для перегляду дерева компонентів, їхніх props і локального state) та **Redux DevTools** (для перегляду вмісту Redux Store у реальному часі, історії відправлених дій (Actions) та можливості «відкату» стану (time-travel debugging));
- **редактор коду Visual Studio Code** з розширеннями для JavaScript/React.

Оскільки лабораторна робота передбачає реалізацію асинхронних CRUD-операцій, необхідне також джерело даних у вигляді REST API. Якщо в попередніх лабораторних роботах курсу такий сервер ще не було розгорнуто, рекомендується використати пакет **json-server**, який за лічені хвилини перетворює звичайний JSON-файл (db.json) на повноцінний REST API з підтримкою методів GET, POST, PUT/PATCH та DELETE. Встановлення виконується командою npm install -g json-server, а запуск — командою json-server --watch db.json --port 3001. Такий підхід дозволяє відпрацювати повноцінний цикл «запит — відповідь» ще до вивчення «справжньої» серверної розробки на ASP.NET Core, яка розглядатиметься в наступному семестрі.

Якщо Node.js ще не встановлено, необхідно завантажити рекомендовану LTS-версію з офіційного сайту https://nodejs.org/ та встановити її, дотримуючись інструкцій інсталятора.

4.4 Порядок виконання роботи і методичні вказівки з її виконання

4.4.1 Теоретична частина

Архітектура Flux та передумови виникнення Redux. У міру зростання React-застосунку та ускладнення дерева компонентів локальний стан (useState) і передавання даних через props перестають бути зручним рішенням: дані, потрібні водночас кільком незалежним компонентам, доводиться «піднімати» (lifting state up) до спільного предка, а звідти — «прокидувати» через props крізь усі проміжні рівні («prop drilling»), навіть якщо самі проміжні компоненти цих даних не використовують. Архітектура **Flux**, запропонована компанією Facebook, вирішує цю проблему через односпрямований потік даних (unidirectional data flow): дія (action) ініціює зміну в централізованому сховищі, а вигляд (view) лише відображає його поточний стан. **Redux** є найпоширенішою бібліотекою, що реалізує ідеї Flux у спрощеному та передбачуваному вигляді.

Основні поняття Redux. У основі Redux лежать чотири взаємопов'язані поняття:

- **Store** — єдиний об'єкт, що зберігає весь стан застосунку та надає методи для отримання поточного стану, відправлення дій (dispatch) і підписки на зміни;
- **State** — сам стан застосунку: як правило, об'єкт із вкладеними полями, кожне з яких відповідає за окрему логічну частину даних;
- **Actions** — прості об'єкти з обов'язковим полем type (що описує подію, яка відбулася) та опціональним полем payload (додаткові дані, потрібні для оновлення стану);

- **Reducers** — чисті функції, що приймають попередній стан і дію, а повертають новий стан, не змінюючи вхідні аргументи та не викликаючи побічних ефектів.

Redux ґрунтується на трьох принципах: **єдине джерело правди** (весь стан зберігається в одному Store), **стан лише для читання** (змінити стан можна лише через відправлення дії) та **зміни виконуються через чисті функції** (reducer обчислює новий стан детерміновано, без побічних ефектів).

Інтеграція Redux з React через <Provider>. Бібліотека react-redux забезпечує зв'язок між Redux Store і деревом React-компонентів. Компонент <Provider> обгортає кореневий компонент застосунку (як правило, у файлі index.js) і передає Store в контекст React, завдяки чому будь-який вкладений компонент отримує до нього доступ без явного передавання через props.

Хуки useSelector та useDispatch. Для роботи зі станом Redux безпосередньо у функціональних компонентах react-redux надає два хуки:

- useSelector(selectorFn) — приймає функцію-селектор і повертає обчислене нею значення з глобального стану; компонент автоматично повторно рендериться, коли відповідна частина стану змінюється;
- useDispatch() — повертає функцію dispatch, за допомогою якої компонент може відправляти дії до Store (наприклад, у відповідь на клік користувача), ініціюючи зміну стану.

Redux Toolkit: configureStore та патерн Slice. Написання Redux «вручну» (окремі константи типів дій, action creators, reducer із конструкцією switch, ручне підключення middleware) є громіздким і схильним до помилок. **Redux Toolkit (RTK)** — офіційна бібліотека, що суттєво спрощує цей процес:

- configureStore замінює класичну функцію createStore: автоматично підключає необхідні middleware (зокрема Thunk), об'єднує кілька reducer'ів в один і вмикає інтеграцію з Redux DevTools без додаткового налаштування;
- createSlice реалізує патерн **Redux Slice** — об'єднує в одному об'єкті назву частини стану (name), її початкове значення (initialState) та набір reducer-функцій, для кожної з яких RTK автоматично генерує відповідний action creator. Це дозволяє описувати логіку однієї предметної області (наприклад, «книги» чи «завдання») в одному файлі, а не розносити типи дій, action creators і reducer по різних файлах.

Асинхронні дії: createAsyncThunk та extraReducers. Reducers у Redux мають бути чистими функціями і не можуть самостійно виконувати асинхронні операції (наприклад, HTTP-запити). Для цього Redux Toolkit надає функцію createAsyncThunk, яка на основі переданої асинхронної функції (наприклад, запиту до REST API через fetch чи axios) автоматично створює три типи дій, що відповідають стадіям виконання запиту:

- pending — відправляється одразу після виклику thunk'a, ще до отримання відповіді від сервера (використовується для показу індикатора завантаження);
- fulfilled — відправляється після успішного завершення запиту та містить отримані дані в полі payload;
- rejected — відправляється, якщо під час виконання запиту сталася помилка (наприклад, мережевий збій або код відповіді, відмінний від 2xx).

Оскільки ці три дії генеруються поза межами об'єкта reducers слайсу, їх обробка виконується в окремій секції extraReducers, де для кожного зі станів (pending/fulfilled/rejected) кожного thunk'a описується відповідна зміна стану — типово: встановлення прапорця завантаження, запис отриманих даних або збереження тексту помилки.

Альтернативним, сучаснішим підходом до взаємодії з API в Redux Toolkit є **RTK Query** — окремий інструмент, що автоматизує кешування, повторні запити та інвалідацію даних без необхідності вручну описувати thunk'и та extraReducers. Порівняння цього підходу з класичним Thunk пропонується здобувачам у межах додаткового завдання.

4.4.2 Практична частина

Крок 1. Підготовка джерела даних. Переконатися, що доступний REST API для обраної предметної області (розгорнутий, наприклад, за допомогою json-server), який підтримує щонайменше методи GET /entity, POST /entity та DELETE /entity/:id, де entity — назва колекції обраного варіанта (наприклад, books, tasks, movies).

Крок 2. Встановлення залежностей. У теці наявного React-проєкту (створеного під час попередньої лабораторної роботи) виконати встановлення необхідних пакетів: npm install @reduxjs/toolkit react-redux.

Крок 3. Створення файлу Store. Створити файл src/app/store.js, у якому за допомогою configureStore створити глобальне сховище, підключивши до нього reducer майбутнього слайсу.

Крок 4. Підключення Store через <Provider>. У файлі index.js імпортувати створений store та компонент Provider з react-redux, обгорнувши ним кореневий компонент <App /> — аналогічно до того, як раніше він уже був обгорнутий у <BrowserRouter>.

Крок 5. Створення Slice предметної області. У теці src/features/ створити файл слайсу (наприклад, booksSlice.js) за допомогою createSlice:

- визначити initialState у вигляді об'єкта з полями для масиву сутностей (items: []), стану завантаження (status: 'idle') та тексту помилки (error: null);

- за допомогою createAsyncThunk описати щонайменше три асинхронні дії: отримання всього списку (fetchEntities), додавання нового запису (addEntity) та видалення запису за ідентифікатором (deleteEntity), кожна з яких виконує відповідний HTTP-запит до API з кроку 1;
- у секції extraReducers обробити стани pending, fulfilled і rejected для кожного з трьох thunk'ів, оновлюючи status, items та error відповідно.

Крок 6. Підключення Slice до Store. Імпортувати reducer, згенерований createSlice (властивість .reducer), у файл store.js та передати його в об'єкт reducer функції configureStore.

Крок 7. Рефакторинг компонентів. У компоненті сторінки списку замінити локальний стан useState, що раніше зберігав масив сутностей, на:

- виклик useSelector для отримання масиву сутностей, статусу завантаження та помилки з глобального стану;
- виклик useDispatch та відправлення дії fetchEntities() всередині useEffect під час першого монтажу компонента;
- умовний рендеринг: індикатор завантаження, якщо status === 'loading'; повідомлення про помилку, якщо status === 'failed'; список сутностей — якщо дані успішно отримано.

Аналогічно оновити компонент форми додавання запису та обробник видалення, замінивши прямі зміни локального масиву на dispatch(addEntity(...)) та dispatch(deleteEntity(id)) відповідно.

Крок 8. Перевірка через Redux DevTools. Відкрити розширення Redux DevTools у браузері та переконатися, що всі дії слайсу (pending/fulfilled/rejected для кожного thunk'а) послідовно з'являються в історії дій (Action History) при завантаженні сторінки, додаванні та видаленні запису, а дерево стану (State) коректно відображає поточний вміст сховища.

4.5 Завдання

Кожна бригада (до двох здобувачів) або здобувач індивідуально продовжує роботу над SPA-застосунком свого варіанта, створеним під час виконання лабораторної роботи № 3 на основі предметної області, обраної в лабораторній роботі № 1 (таблиця 4.1 містить той самий перелік предметних областей із тією самою нумерацією, що й таблиці 1.1 та 3.1).

Незалежно від обраного варіанта, реалізований застосунок повинен відповідати таким загальним вимогам:

- керування колекцією основних сутностей варіанта повністю перенесено з локального стану (useState) на глобальний стан Redux, реалізований за допомогою createSlice;

- створено щонайменше один слайс предметної області з initialState, полями стану завантаження (status) та помилки (error);
- реалізовано щонайменше три асинхронні дії через createAsyncThunk: отримання списку сутностей, додавання нового запису та видалення запису за ідентифікатором, кожна з яких звертається до REST API;
- у секції extraReducers коректно оброблено стани pending, fulfilled і rejected для кожної асинхронної дії;
- компоненти, що відображають список і деталі сутностей, отримують дані виключно через useSelector, а ініціюють зміни — виключно через useDispatch;
- інтерфейс відображає індикатор завантаження під час виконання запиту та повідомлення про помилку в разі невдалого запиту;
- маршрутизація (React Router), реалізована у попередній лабораторній роботі, зберігається без змін — предметом цієї роботи є виключно управління станом.

Таблиця 4.1 – Основне завдання для лабораторної роботи № 4

Варіант і назва	**Завдання**
Варіант 1. Система управління бібліотекою	Перенести управління списком книг у Redux: слайс booksSlice зі станом { items: [], status, error }; асинхронні дії fetchBooks (GET /books), addBook (POST /books), deleteBook (DELETE /books/:id); компоненти BookList і BookForm перевести на useSelector/useDispatch.
Варіант 2. Менеджер завдань (To-Do List)	Перенести управління списком завдань у Redux: слайс tasksSlice; асинхронні дії fetchTasks (GET /tasks), addTask (POST /tasks), deleteTask (DELETE /tasks/:id); компоненти TaskList і TaskForm перевести на useSelector/useDispatch.
Варіант 3. Система обліку студентів	Перенести управління списком студентів у Redux: слайс studentsSlice; асинхронні дії fetchStudents (GET /students), addStudent (POST /students), deleteStudent (DELETE /students/:id); компоненти StudentList і StudentForm перевести на useSelector/useDispatch.
Варіант 4. Інтерактивний словник	Перенести управління списком слів у Redux: слайс wordsSlice; асинхронні дії fetchWords (GET /words), addWord (POST /words), deleteWord (DELETE /words/:id); компоненти WordList і WordForm перевести на useSelector/useDispatch.
Варіант 5. Калькулятор витрат	Перенести управління списком витрат у Redux: слайс expensesSlice; асинхронні дії fetchExpenses (GET

Варіант і назва	Завдання
	/expenses), addExpense (POST /expenses), deleteExpense (DELETE /expenses/:id); компоненти ExpenseList і ExpenseForm перевести на useSelector/useDispatch.
Варіант 6. Аналізатор тексту	Перенести управління історією аналізів у Redux: слайс historySlice; асинхронні дії fetchHistory (GET /history), addHistoryEntry (POST /history), deleteHistoryEntry (DELETE /history/:id); компоненти HistoryList і TextAnalyzerForm перевести на useSelector/useDispatch.
Варіант 7. Галерея зображень	Перенести управління колекцією зображень у Redux: слайс imagesSlice; асинхронні дії fetchImages (GET /images), addImage (POST /images), deleteImage (DELETE /images/:id); компоненти Gallery і ImageForm перевести на useSelector/useDispatch.
Варіант 8. Калькулятор площі фігур	Перенести управління історією розрахунків у Redux: слайс historySlice; асинхронні дії fetchHistory (GET /history), addCalculation (POST /history), deleteHistoryEntry (DELETE /history/:id); компоненти HistoryList і ShapeForm перевести на useSelector/useDispatch.
Варіант 9. Система тестування	Перенести управління списком тестів у Redux: слайс testsSlice; асинхронні дії fetchTests (GET /tests), addTest (POST /tests), deleteTest (DELETE /tests/:id); компоненти TestList і TestRunner перевести на useSelector/useDispatch.
Варіант 10. Система управління контактами	Перенести управління списком контактів у Redux: слайс contactsSlice; асинхронні дії fetchContacts (GET /contacts), addContact (POST /contacts), deleteContact (DELETE /contacts/:id); компоненти ContactList і ContactForm перевести на useSelector/useDispatch.
Варіант 11. Каталог фільмів та серіалів	Перенести управління каталогом фільмів у Redux: слайс moviesSlice; асинхронні дії fetchMovies (GET /movies), addMovie (POST /movies), deleteMovie (DELETE /movies/:id); компоненти MovieList і MovieForm перевести на useSelector/useDispatch.
Варіант 12. Трекер щоденних звичок	Перенести управління списком звичок у Redux: слайс habitsSlice; асинхронні дії fetchHabits (GET /habits), addHabit (POST /habits), deleteHabit (DELETE /habits/:id); компоненти HabitList і HabitForm перевести на useSelector/useDispatch.

Варіант і назва	Завдання
Варіант 13. Система бронювання столиків у ресторані	Перенести управління списком бронювань у Redux: слайс reservationsSlice; асинхронні дії fetchReservations (GET /reservations), addReservation (POST /reservations), deleteReservation (DELETE /reservations/:id); компоненти ReservationList і ReservationForm перевести на useSelector/useDispatch.
Варіант 14. Міні-платформа для публікації статей (блог)	Перенести управління списком статей у Redux: слайс articlesSlice; асинхронні дії fetchArticles (GET /articles), addArticle (POST /articles), deleteArticle (DELETE /articles/:id); компоненти ArticleList і ArticleForm перевести на useSelector/useDispatch.
Варіант 15. Щоденник фітнес-тренувань	Перенести управління списком тренувань у Redux: слайс workoutsSlice; асинхронні дії fetchWorkouts (GET /workouts), addWorkout (POST /workouts), deleteWorkout (DELETE /workouts/:id); компоненти WorkoutList і WorkoutForm перевести на useSelector/useDispatch.
Варіант16. Каталог рецептів	Перенести управління списком рецептів у Redux: слайс recipesSlice; асинхронні дії fetchRecipes (GET /recipes), addRecipe (POST /recipes), deleteRecipe (DELETE /recipes/:id); компоненти RecipeList і RecipeForm перевести на useSelector/useDispatch.
Варіант 17. Планувальник подій	Перенести управління списком подій у Redux: слайс eventsSlice; асинхронні дії fetchEvents (GET /events), addEvent (POST /events), deleteEvent (DELETE /events/:id); компоненти EventList і EventForm перевести на useSelector/useDispatch.
Варіант 18. Трекер вакансій (пошук роботи)	Перенести управління списком заявок у Redux: слайс applicationsSlice; асинхронні дії fetchApplications (GET /applications), addApplication (POST /applications), deleteApplication (DELETE /applications/:id); компоненти ApplicationList і ApplicationForm перевести на useSelector/useDispatch.
Варіант 19. Облік клієнтів та тварин ветеринарної клініки	Перенести управління списком карток тварин у Redux: слайс petsSlice; асинхронні дії fetchPets (GET /pets), addPet (POST /pets), deletePet (DELETE /pets/:id); компоненти PetList і PetForm перевести на useSelector/useDispatch.

Варіант і назва	Завдання
Варіант 20. Персональний нотатник (щоденник)	Перенести управління списком нотаток у Redux: слайс notesSlice; асинхронні дії fetchNotes (GET /notes), addNote (POST /notes), deleteNote (DELETE /notes/:id); компоненти NoteList і NoteForm перевести на useSelector/useDispatch.

Здобувач також може за погодженням із викладачем продовжити роботу над власною предметною областю за умови збереження всіх загальних вимог, наведених вище.

Додаткове завдання виконується здобувачем за бажанням, понад основний обсяг роботи, і оцінюється додатковими балами на високу оцінку. Можна обрати один або кілька варіантів із наведених нижче.

Таблиця 4.2 – Додаткове завдання для лабораторної роботи № 4

Варіант і назва	Завдання
Варіант 1. Власний Middleware для логування дій	Реалізувати власний Redux middleware (функцію виду store => next => action => {...}), яка виводить у консоль браузера тип кожної відправленої дії, її payload, а також стан сховища до та після її обробки. Підключити middleware до Store через параметр middleware функції configureStore, скориставшись getDefaultMiddleware().concat(loggerMiddleware).
Варіант 2. Перехід на RTK Query	Переробити взаємодію з API для одного зі слайсів, замінивши createAsyncThunk на RTK Query: створити API-сервіс за допомогою createApi з описом ендпоінтів (getEntities, addEntity, deleteEntity) і використати автоматично згенеровані хуки (наприклад, useGetEntitiesQuery, useAddEntityMutation) безпосередньо в компонентах. Порівняти отриманий підхід із варіантом на основі Thunk за обсягом коду та наявністю автоматичного кешування й повторних запитів (refetching).
Варіант 3. Мемоізовані селектори через createSelector	За допомогою функції createSelector реалізувати щонайменше один похідний (обчислюваний) селектор — наприклад, відфільтрований чи відсортований список сутностей або агреговане значення (кількість записів, сума, середнє значення). Продемонструвати, що такий мемоізований селектор не спричиняє повторного обчислення, якщо вхідні дані не змінилися, на відміну від обчислення безпосередньо в тілі компонента.

Варіант і назва	Завдання
Варіант 4. Оптимістичні оновлення (Optimistic Updates)	Реалізувати оптимістичне видалення запису: елемент миттєво зникає зі списку в інтерфейсі одразу після дії користувача, ще до отримання відповіді від сервера, а сам запит на видалення виконується у фоновому режимі. У разі помилки запиту — повернути видалений елемент назад у стан і показати користувачу відповідне повідомлення.
Варіант 5. Збереження стану через redux-persist	Підключити бібліотеку redux-persist для автоматичного збереження частини стану Redux (наприклад, кешованого списку сутностей або обраних фільтрів) у localStorage браузера між перезавантаженнями сторінки. Налаштувати persistStore і persistReducer, а також обгорнути кореневий компонент застосунку в <PersistGate>.
Варіант 6. Нормалізація стану через createEntityAdapter	Переробити слайс, використавши createEntityAdapter замість звичайного масиву items: нормалізувати колекцію сутностей у вигляді { ids: [], entities: {} } та скористатися згенерованими селекторами (selectAll, selectById) і методами адаптера (setAll, addOne, removeOne) в extraReducers. Порівняти зручність доступу до окремого запису за ідентифікатором до і після нормалізації.
Варіант 7. Дебаунсований пошук із синхронізацією в Redux	Додати до слайсу поле searchQuery та відповідний синхронний reducer для його оновлення. Реалізувати власний хук (наприклад, useDebounce), який затримує диспетчеризацію дії оновлення пошукового запиту на 300–500 мс після останнього натискання клавіші, і використати значення searchQuery зі стану для фільтрації списку сутностей, що відображається.
Варіант 8. Скасування останньої дії (Undo)	Реалізувати можливість скасування останнього видалення запису: перед фактичним видаленням зберігати копію запису в окремому полі стану (наприклад, lastDeleted), а компонент — показувати тимчасове повідомлення з кнопкою «Скасувати», що диспетчеризує дію повернення запису назад у список протягом кількох секунд після видалення.
Варіант 9. Пагінація списку на рівні Redux	Розширити асинхронну дію отримання списку (fetchEntities) параметром номера сторінки (наприклад, fetchEntities(page)), що передається у відповідний query-параметр запиту до API (GET /entity?_page=...&_limit=...). Зберігати в стані слайсу поточну сторінку та загальну кількість записів, а в

Варіант і назва	Завдання
	інтерфейсі — реалізувати елементи керування переходом між сторінками.
Варіант 10. Модульне тестування Redux-логіки	За допомогою Jest написати щонайменше три модульні тести для створеного слайсу: перевірку початкового стану (initialState), перевірку коректної зміни стану синхронним reducer'ом на конкретному прикладі дії та перевірку зміни стану при обробці станів pending/fulfilled/rejected одного з асинхронних thunk'ів (з використанням mock-даних замість реального запиту до API).

4.6 Зміст звіту

За результатами виконання лабораторної роботи здобувачі повинні підготувати звіт, що містить повну інформацію про хід виконання завдання та отримані результати.

Першим елементом звіту є титульна сторінка, на якій зазначаються повна назва навчального закладу, назва дисципліни, назва лабораторної роботи, прізвище та ім'я здобувача, а також номер академічної групи.

Далі подається розділ «Мета роботи», де стисло формулюється основна ціль лабораторної роботи та вказується обраний варіант завдання.

У розділі «Хід роботи» необхідно навести:

- лістинг конфігурації Store (файл store.js з викликом configureStore);
- лістинг створеного слайсу предметної області: initialState, синхронні reducers (за наявності), асинхронні дії, оголошені через createAsyncThunk, та секцію extraReducers;
- лістинг оновлених компонентів, у яких useState замінено на useSelector/useDispatch, із коротким поясненням, яку саме частину логіки було рефакторено.
- Обов'язковою частиною розділу є **скріншоти розширення Redux DevTools**, що демонструють:
- дерево стану (State) застосунку після завантаження списку сутностей;
- історію відправлених дій (Action History), що містить послідовність pending → fulfilled (або rejected) щонайменше для однієї з реалізованих асинхронних дій.

Завершальним розділом є висновки, у яких здобувач узагальнює отримані результати, оцінює відповідність реалізації поставленим вимогам, описує труднощі, з якими зіткнувся під час переходу від локального стану до Redux чи налаштування асинхронних дій, а також описує виконане (за наявності) додаткове завдання.

4.7 Контрольні запитання і завдання

1. Що таке Redux і яку проблему він вирішує в React-застосунках порівняно з передаванням даних через props?
2. Сформулюйте три принципи Redux. Що означає принцип «стан лише для читання»?
3. Що таке Store, Action та Reducer? Яка роль кожного з цих елементів в архітектурі Redux?
4. У чому полягають основні відмінності Redux Toolkit від «класичного» Redux?
5. Яке призначення функції createSlice? З яких основних частин складається об'єкт, що передається в неї?
6. Яку роль виконує компонент Provider у React-застосунку, що використовує Redux?
7. Чим відрізняються хуки useSelector та useDispatch? Наведіть приклад типового використання кожного з них.
8. Що таке createAsyncThunk і які три стани життєвого циклу асинхронного запиту він автоматично генерує?
9. Яке призначення секції extraReducers у Redux Slice? Чому дії, згенеровані createAsyncThunk, обробляються саме там, а не у звичайному об'єкті reducers?
10. Що таке middleware в Redux? Яку роль виконує Thunk-middleware і яким чином він підключається за допомогою configureStore?

Електронне навчальне видання

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторних робіт з дисципліни
«ТЕХНОЛОГІЇ ВЕБ-РОЗРОБКИ»

для здобувачів 1-го (бакалаврського) рівня вищої освіти
спеціальності F7 «Комп'ютерна інженерія»
освітньої програми «Комп'ютерна інженерія»

Упорядник БЕЗРОДНИЙ Владислав В'ячеславович

Відповідальний випусковий С.В. Чумаченко

Редактор

Комп'ютерна верстка

План 2026(_____ півріччя), поз. ____
Підп. до використання ___.___.20___. Формат pdf Обсяг даних 3,75 Мб.

ХНУРЕ, 61166, Харків, просп. Науки, 14, E-mail: info@nure.ua

Підготовлено в редакційно-видавничому відділі ХНУРЕ
Свідоцтво суб'єкта видавничої справи ДК №1409 від 26.06.2003